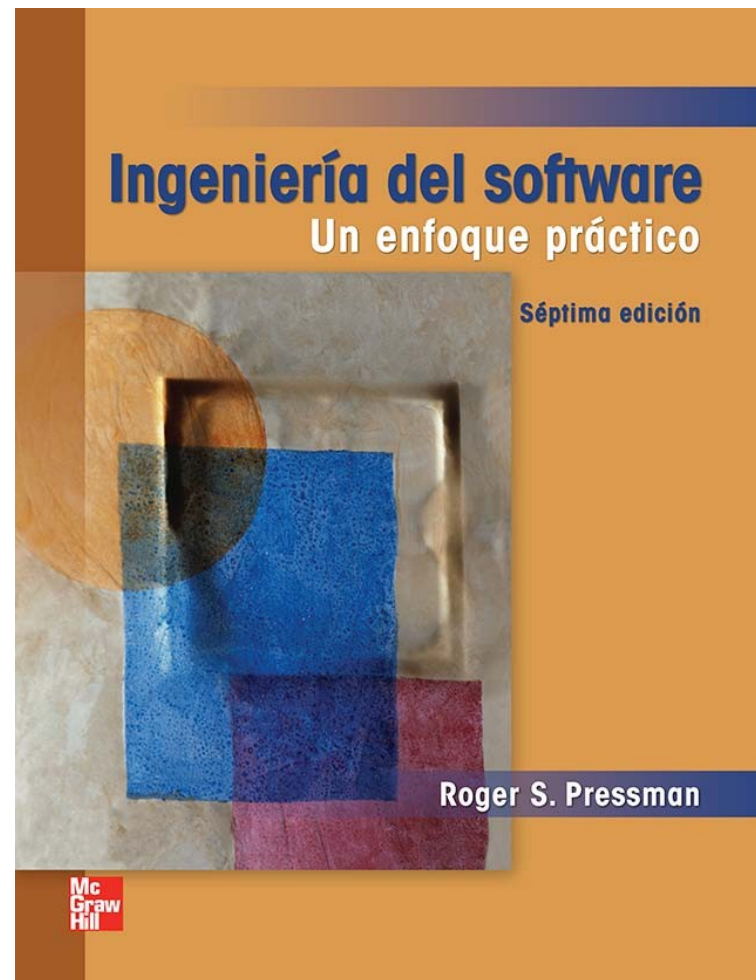


Ingeniería de Software

Olaf Reitmaier
olafrv@gmail.com
Junio de 2018

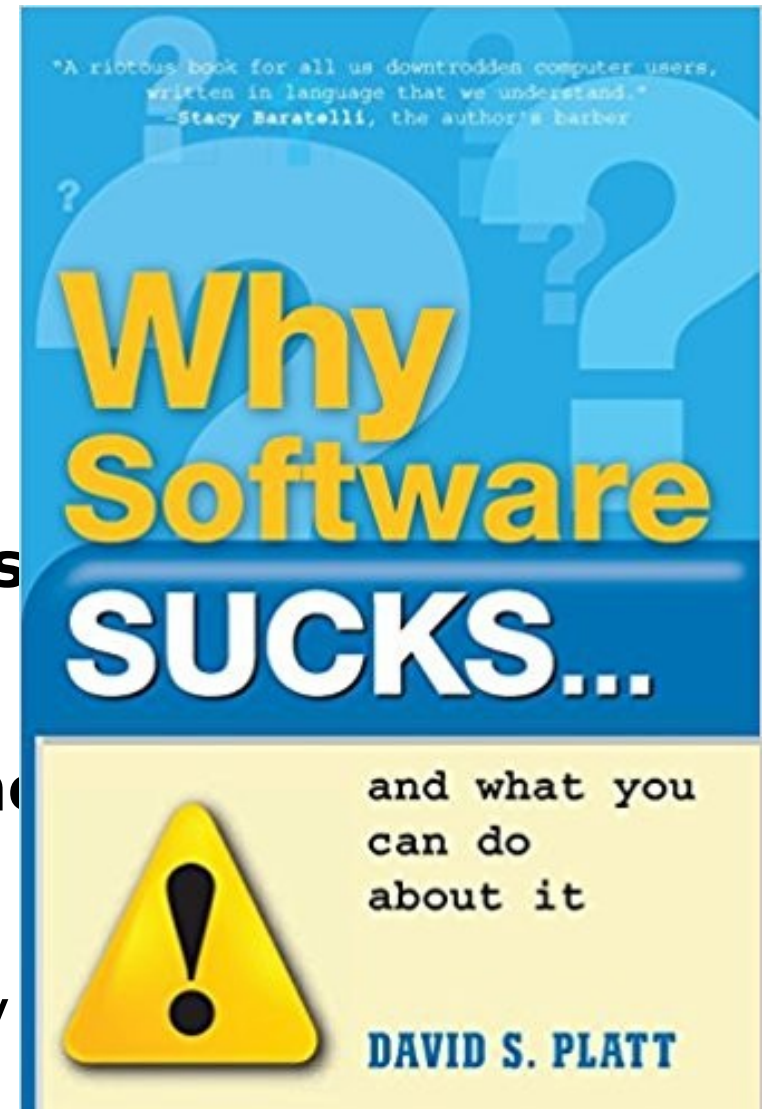
Ingeniería de Software



Problema del Software

¿Por qué sucede lo siguiente?

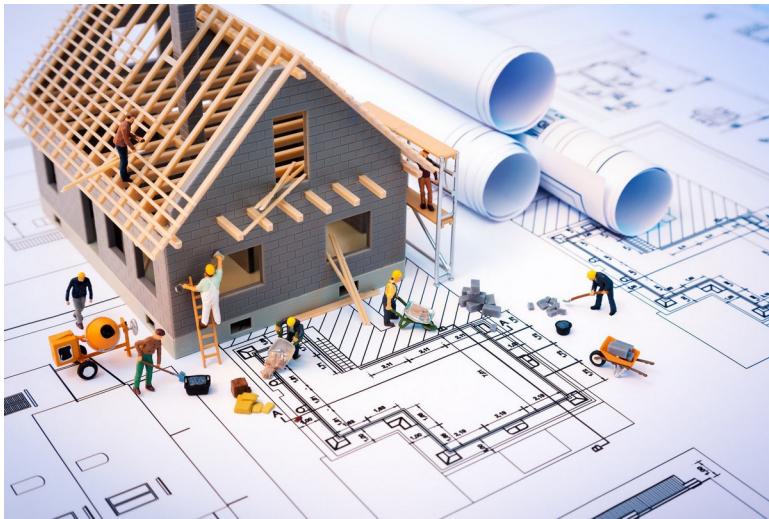
- Demasiado **tiempo y esfuerzo**:
 - **Terminarlo**
 - **Mantenerlo**
- No se detectan todos los **errores** antes de **entregarlo**
- Dificultades para **medir el avance** del desarrollo
- **Altos los costos** de desarrollo y mantenimiento



Software: ~~Proyecto~~ Proceso

El software se desarrolla y mantiene con **intelecto**, un recurso muy **dinámico y volátil** que **evoluciona durante su uso**.

Esto implica que para los proyectos de desarrollo de software **es insuficiente** gestionarlos como **proyectos tradicionales**.

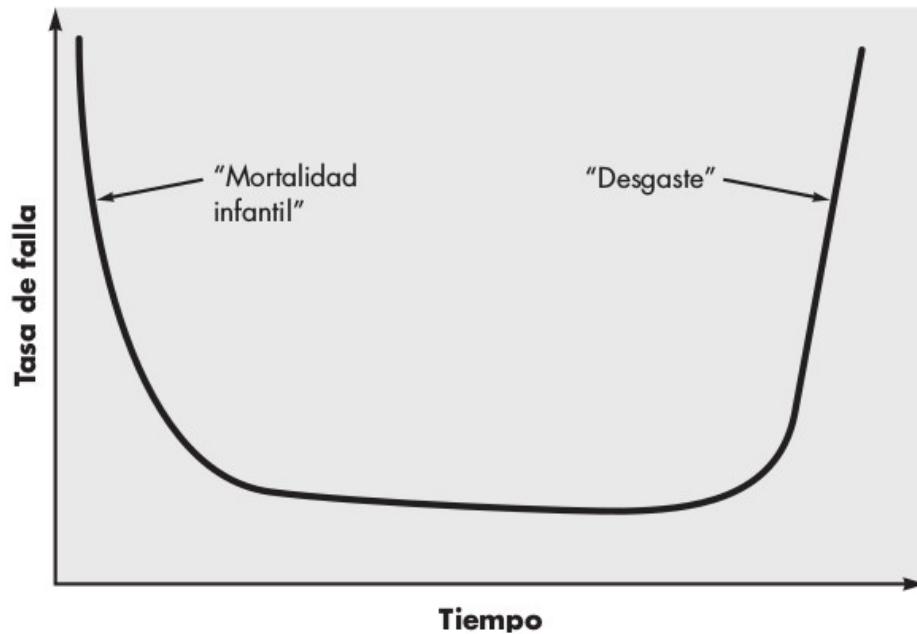


Hardware



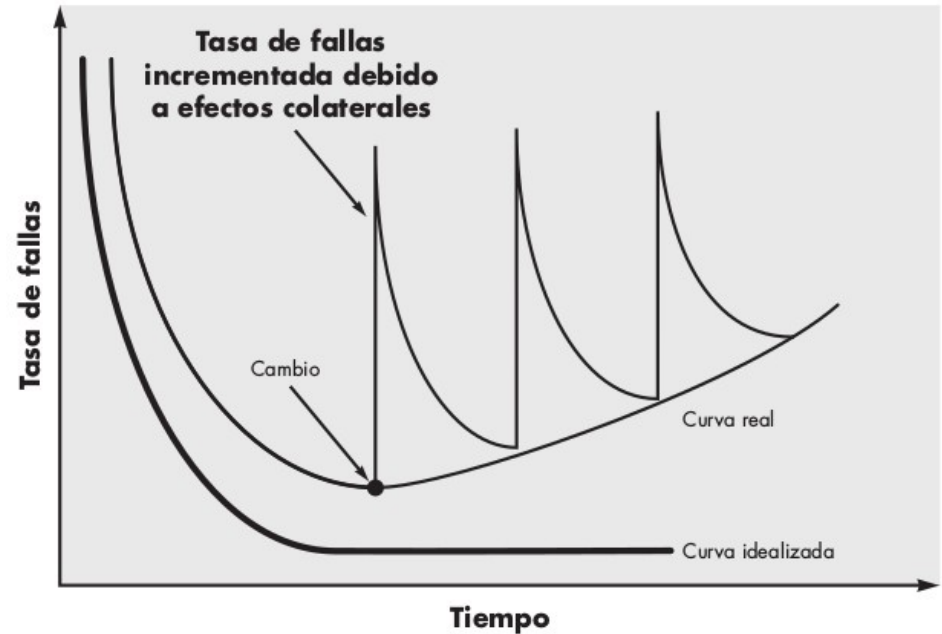
Software

Software: ~~Desgaste~~ Deterioro



Fallas de Hardware

(Fábrica y/o Electronicos y Mecánicos)



Fallas de Software

(Cambios Internos y Externos)

Software: ¿Reusabilidad?

La mayor parte del software se construye para un **uso individualizado y no orientado a componentes**.

Un **componente de software** debe diseñarse e implementarse de modo que pueda **reusarse sin mayor esfuerzo** en una amplia gamma de software diferente.



Software: Legado (Heredado)

- Los sistemas de **software legado**:

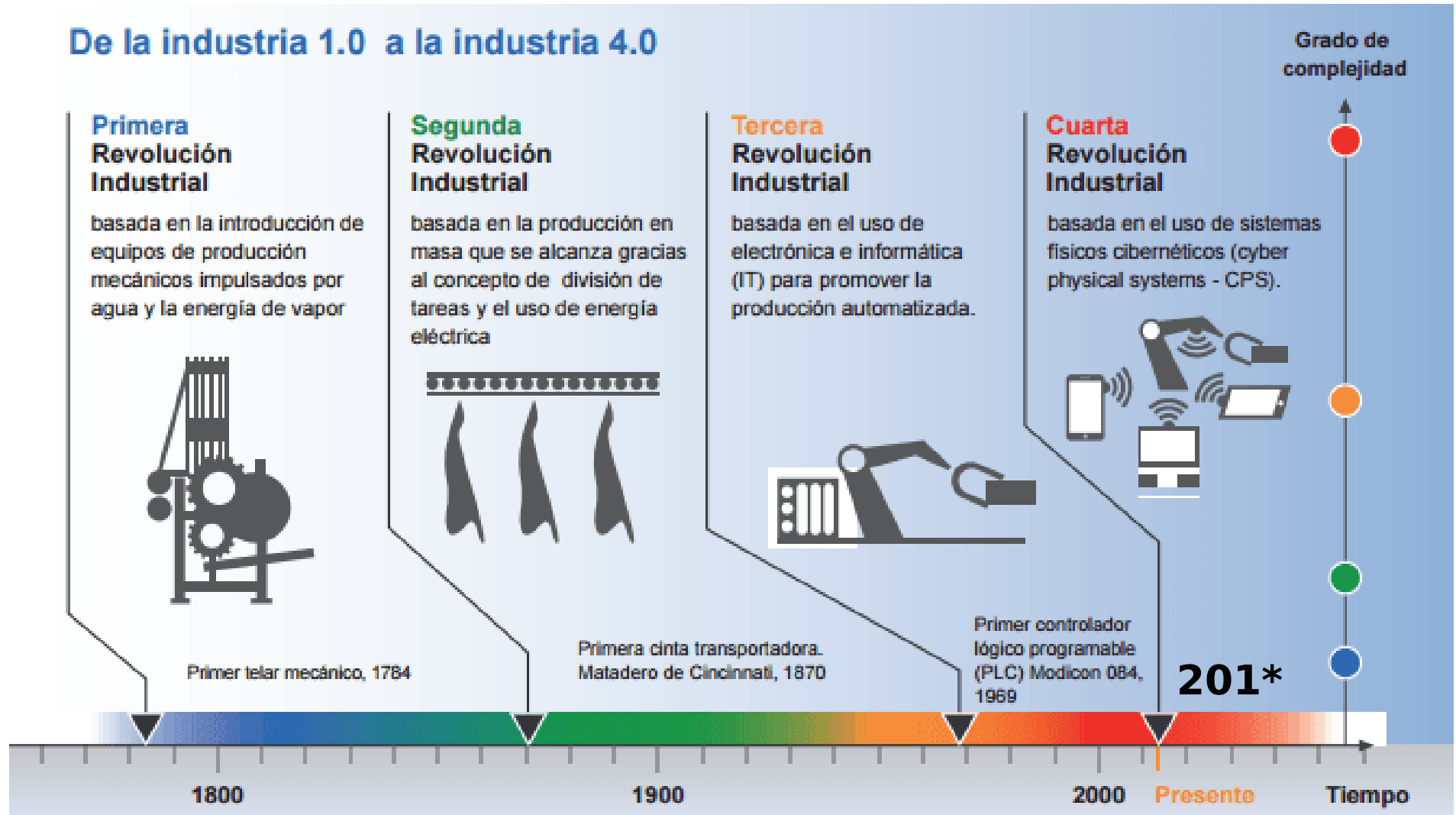
- Desarrollados hace (una) década(s)
- Modificados para cumplir al negocio
- Causa de dolores de cabeza a muchos
- Riesgosos de mantener y evolucionar
- Costosos y algunos de mala calidad



- La única **respuesta razonable** es: *hacer nada o hacer reingeniería* para que sea **viable en el futuro**.
- La **IS moderna** tiene como meta **desarrollar metodologías** que permitan al software una **evolución sin traumas**.

Software: Industria 4.0

De la industria 1.0 a la industria 4.0



Ingeniería de Software

- 1) **Aplicación de la ingeniería al software:** enfoque sistemático, disciplinado y cuantificable al desarrollo y mantenimiento de software.
- 2) **Estudio de los enfoques posibles.**



Ingenierio de Software

- **Complejidad**
 - **Simplicidad**
- **Incetidumbre**
 - **Adaptabilidad**
- **Volatilidad**
 - **Agilidad**
- **Ambigüedad**
 - **Ritualidad**



Ingenierio de Software

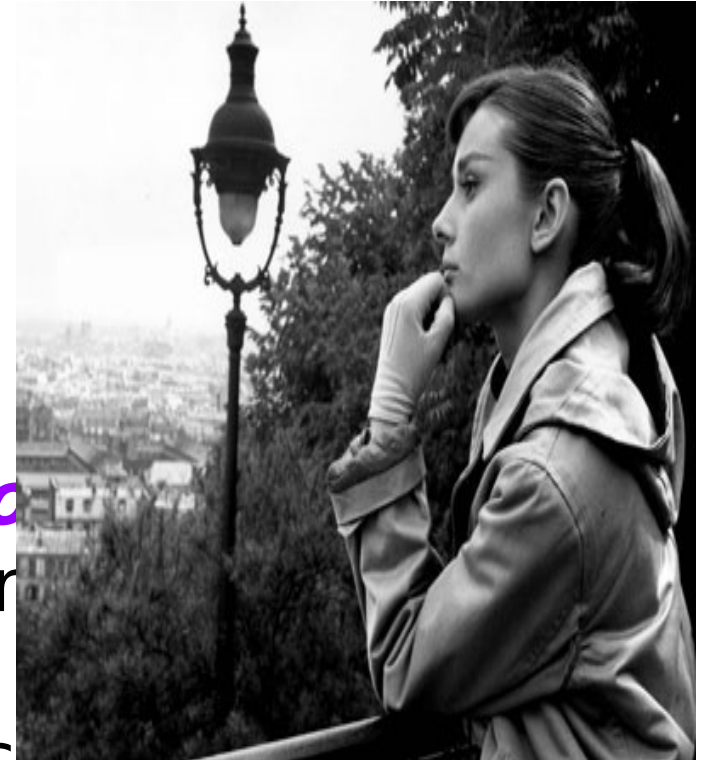
- Simple
- Adaptable
- Ágil
- Ritualista

- ~~Genio~~
- ~~Solitario~~
- ~~Poderoso~~
- ~~o~~
- ~~o~~



Principios Generales

- Recordar la razón de todo:
 - ***Dar valor a los usuarios!***
- Mantenerlo sencillo...
 - ***No debe ser complejo!***
- Mantener la visión a futuro
 - ***Más allá del requerimiento***
- Recordar que otros lo consumirán
 - ***Debe ser de calidad!***
- Anticipar y planear la reutilización
 - ***Pensar!***



Mitos y Leyendas

- **Metemos más gente y listo (Orda de mongoles)...**
 - Proyecto no tradicional, proceso no mecánico...
- **Es suficiente con el enunciado del requerimiento...**
 - Los detalles los dejamos para después...
- **El único entregable importante es el sistema...**
 - El resto no será necesario por ahora...
- **La ingeniería de software nos retrasará...**
 - Por qué requiere demasiada documentación...

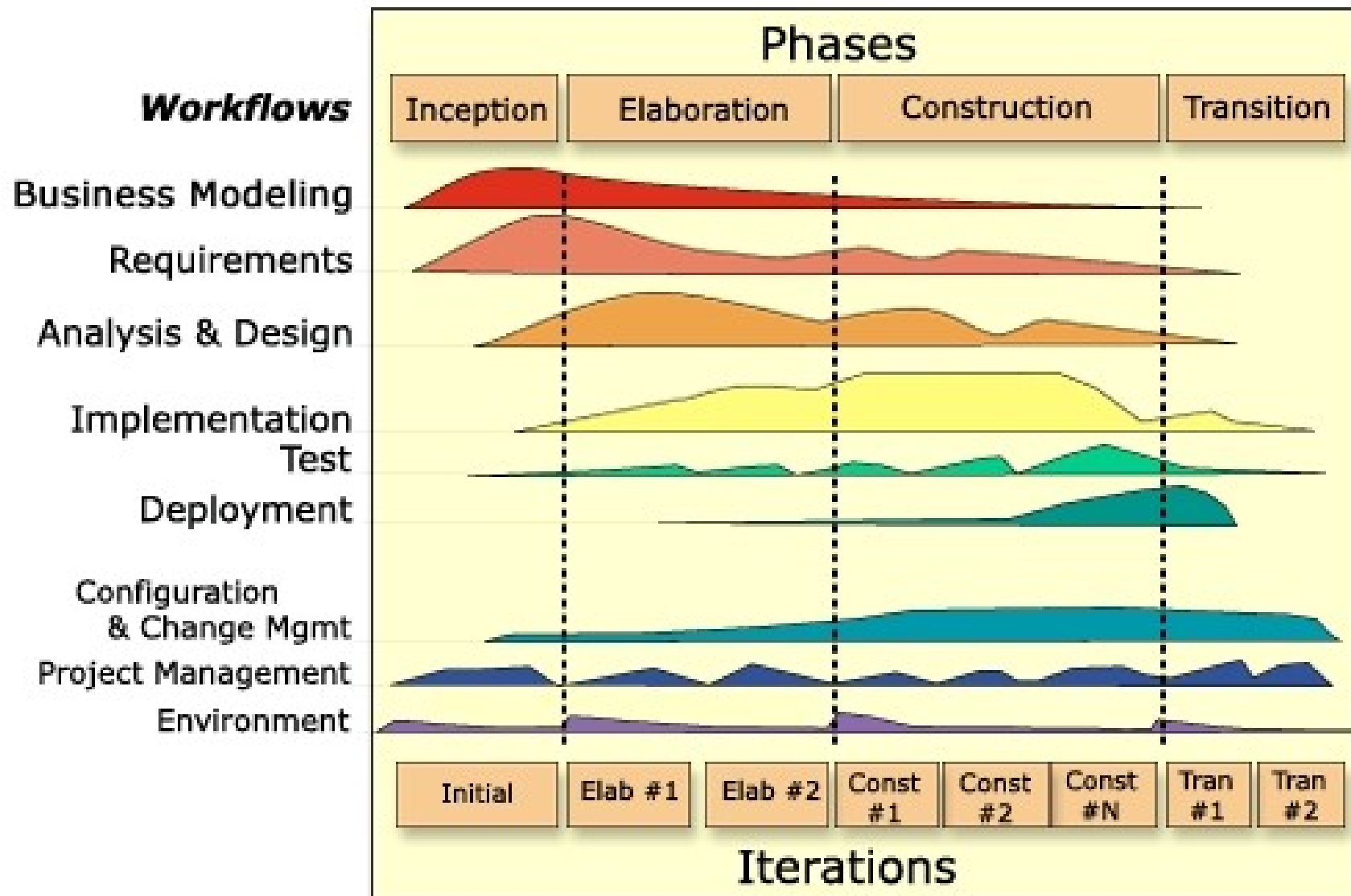
Modelos de Desarrollo

- **Lineales**
- **Iterativos**
- **Incrementales** (Acumulativos)
- **Evolutivos** (Prototipos)
- **Convencionales:** *Cascada, V, UP...*
- **Ágiles:** *AUP, XP, Scrum, OpenUP...*

Modelos Especializados

- Basado en **Métodos Formales:**
 - Uso de Matemáticas
- Basado en **Componentes:**
 - Uso de Fragmentos Prefabricados
 - **Incremental, Iterativo y Evolutivo**
- Basado en **Aspectos:**
 - Enfoque tema(s) específico(s)
 - Seguridad, Tolerancia a Fallos, ...

Unified Process (Rational/IBM)



Enterprise UP (Agile UP Cons.)

Development Disciplines

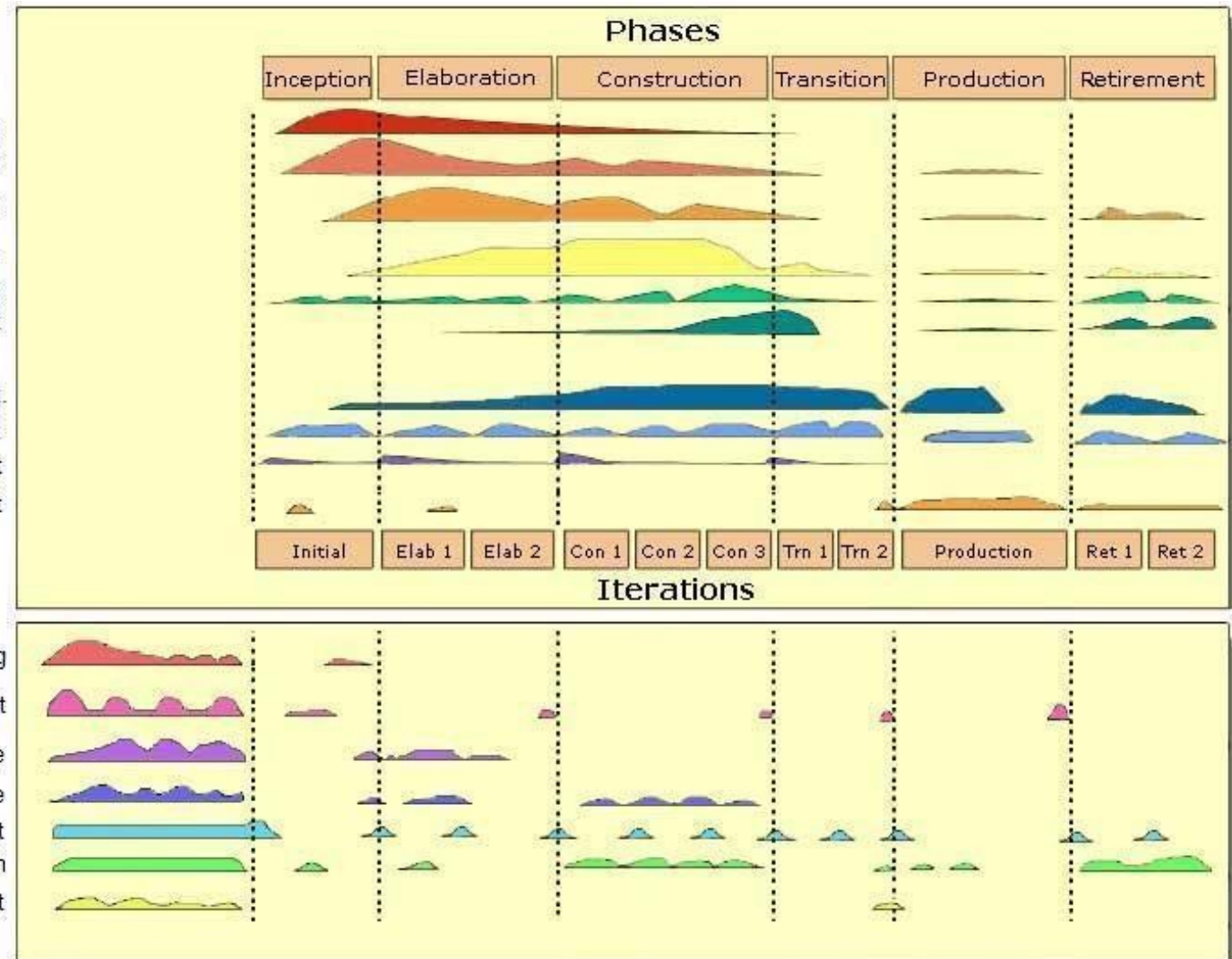
Business Modeling
Requirements
Analysis & Design
Implementation
Test
Deployment

Support Disciplines

Configuration and Change Mgmt.
Project Management
Environment
Operations & Support

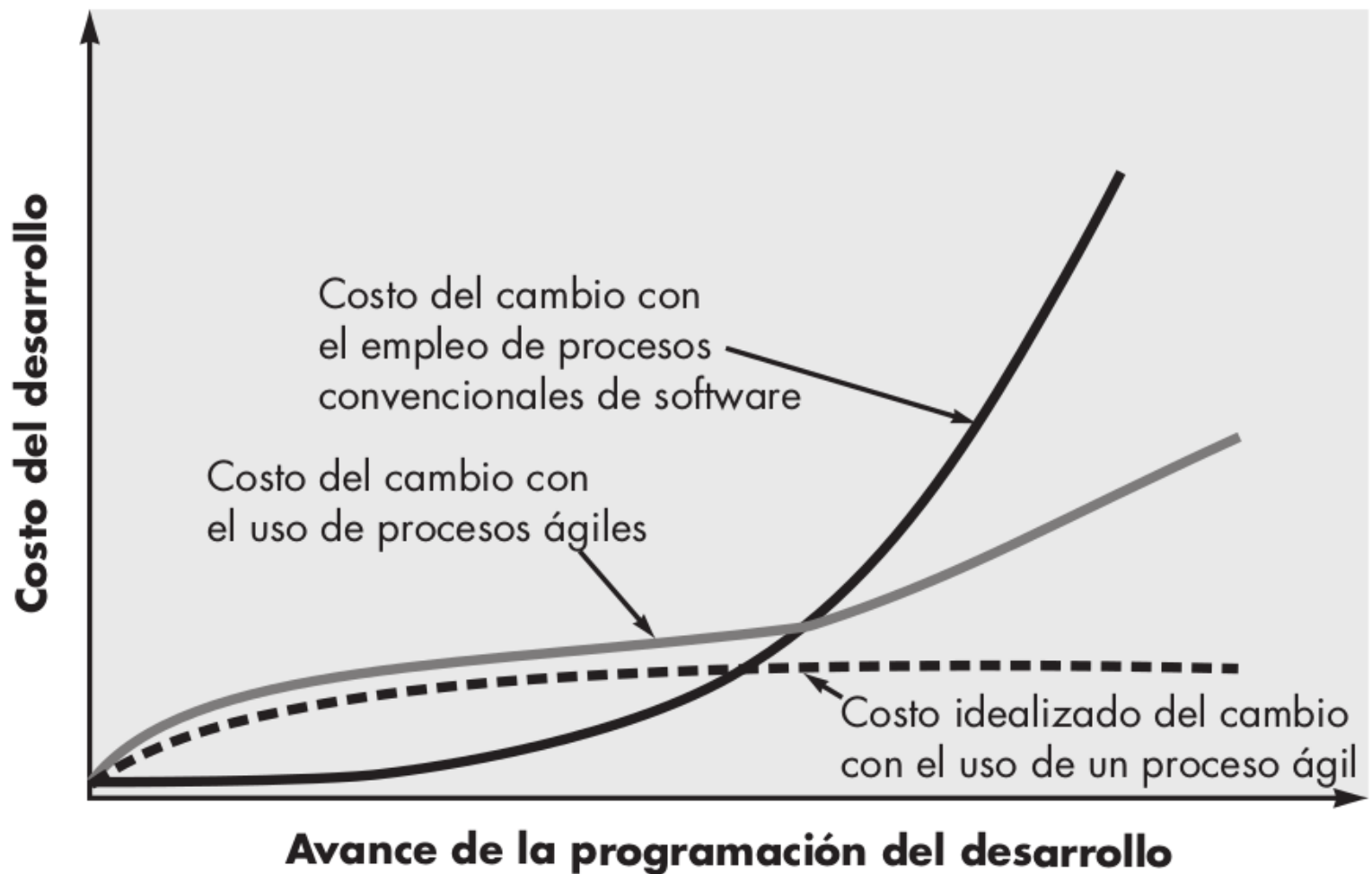
Enterprise Disciplines

Enterprise Business Modeling
Portfolio Management
Enterprise Architecture
Strategic Reuse
People Management
Enterprise Administration
Software Process Improvement

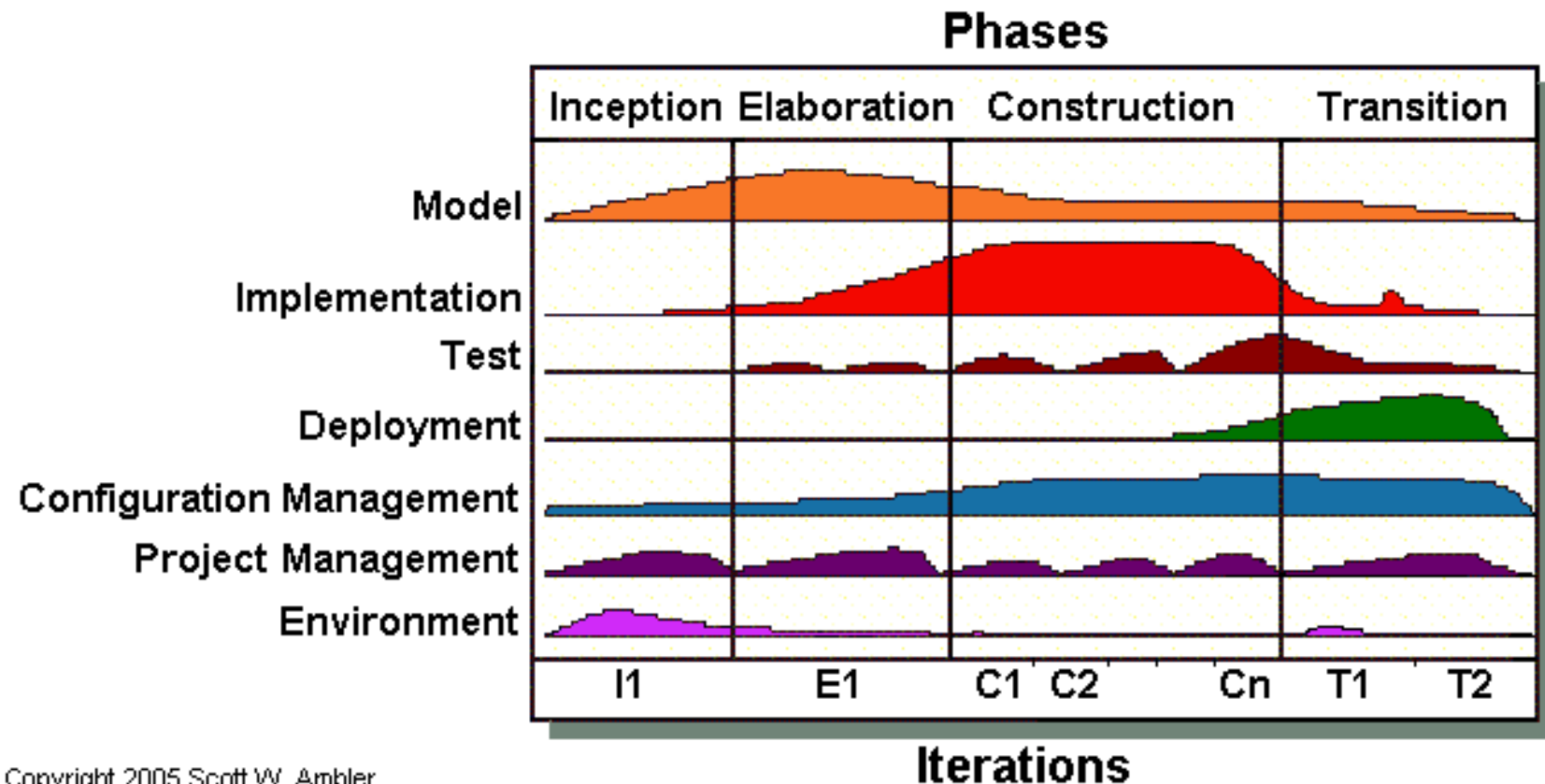


Copyright 2003-2005 Scott W. Ambler

Agilidad del Desarrollo



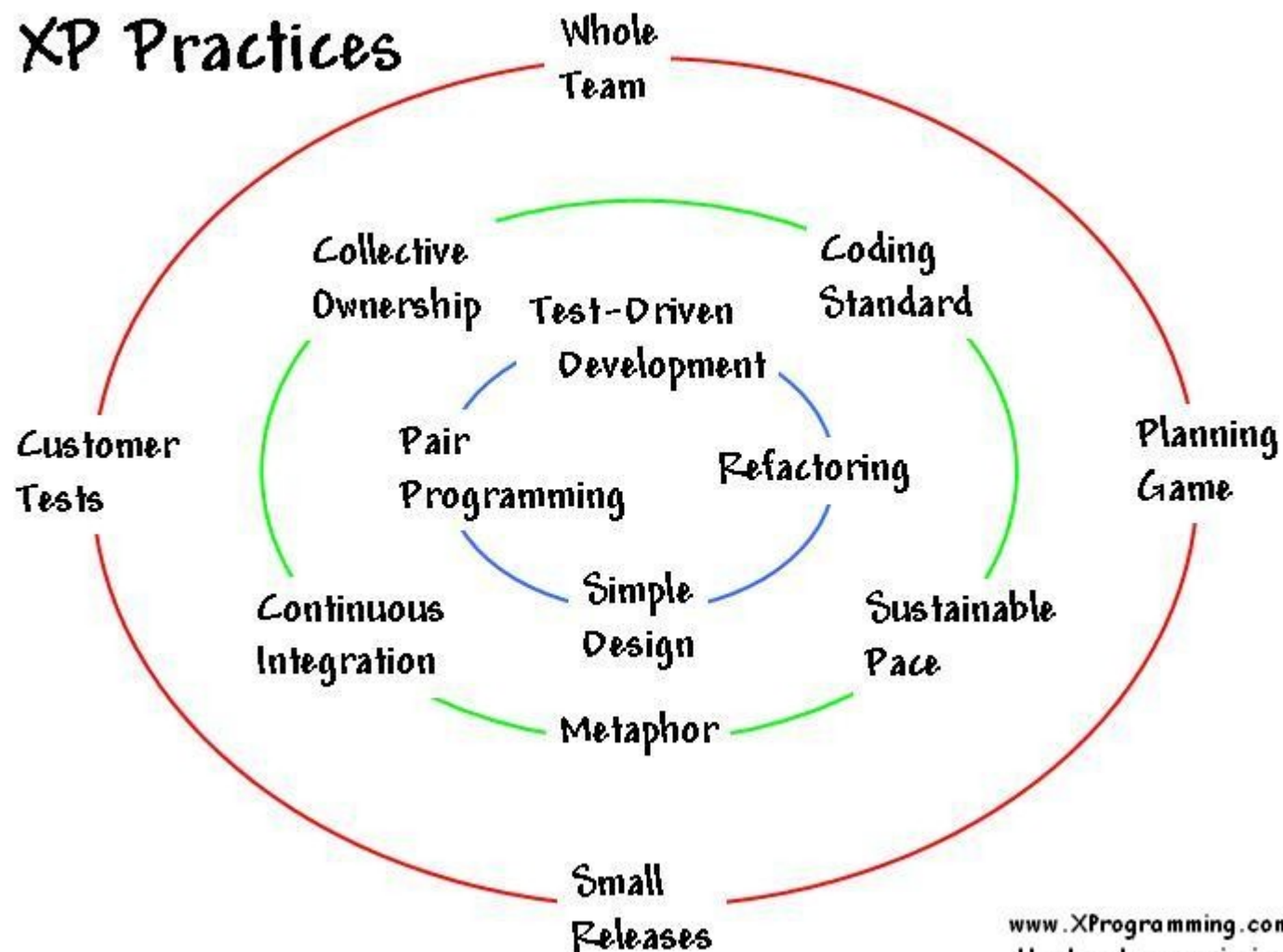
Agile Unified Process (AUP)



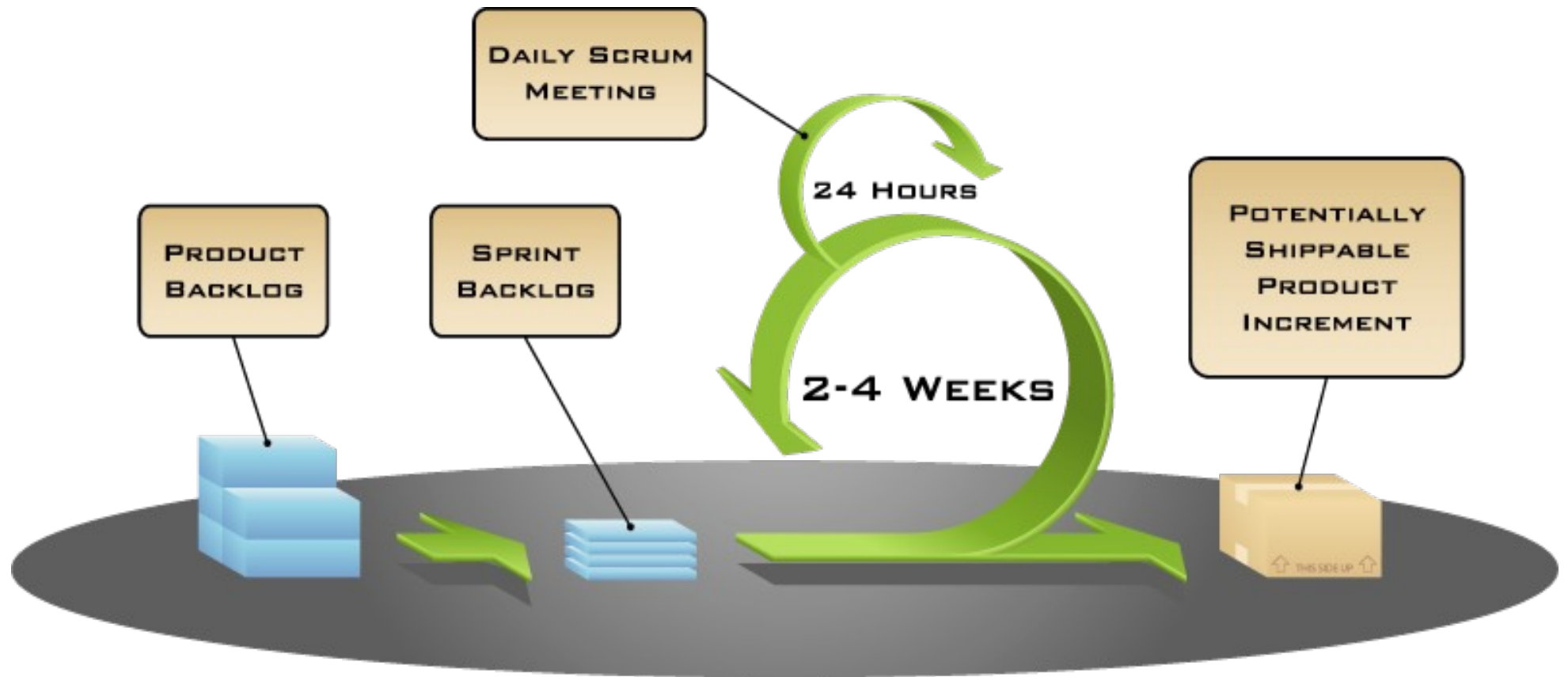
Copyright 2005 Scott W. Ambler

<http://www.ambysoft.com/unifiedprocess/agileUP.html>

Extreme Programming (XP)



Scrum (*del Rugby*)

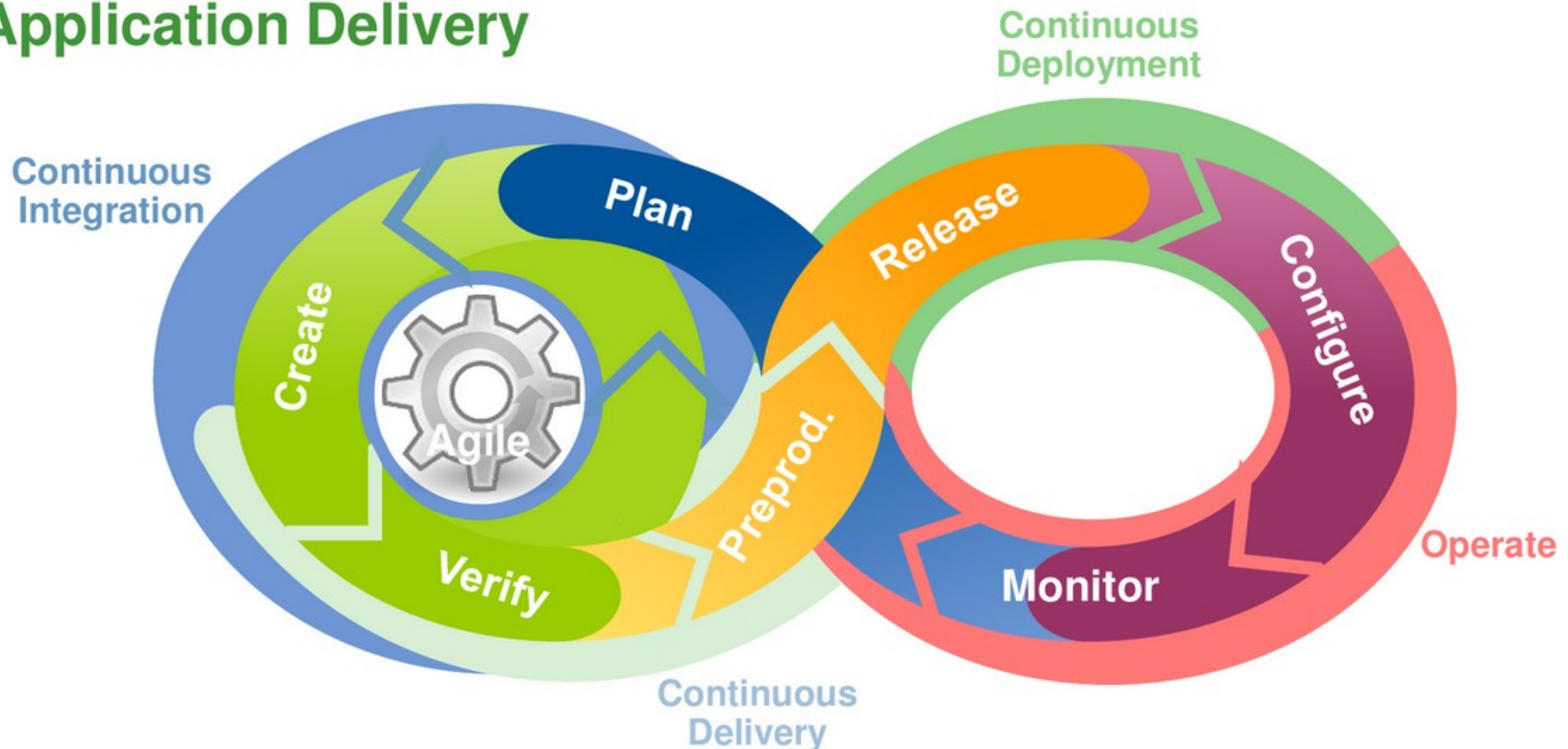


COPYRIGHT © 2005, MOUNTAIN GOAT SOFTWARE

Gartner DevOps Toolchain

DevOps Toolchain Continuous Application Delivery

Step 7 @ the core of change

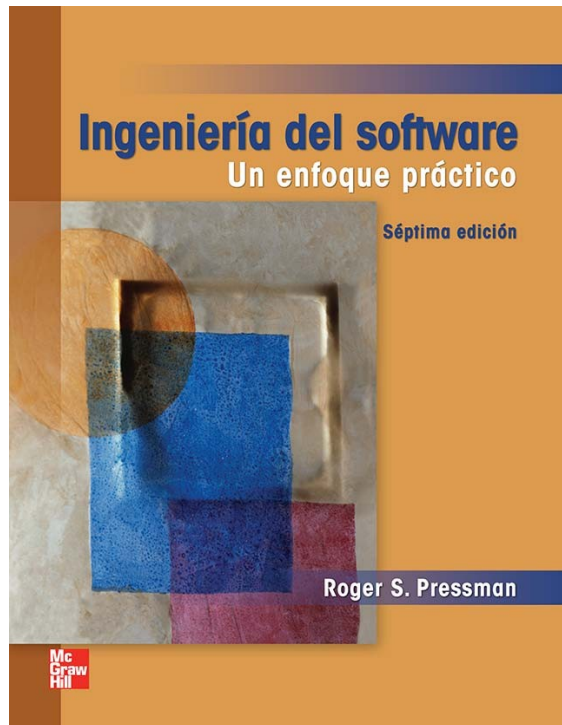


Ingeniería de Software



OpenUP
Agile

DevOps
Continuous
Delivery



*tecnológica***gente**

Gartner DevOps Toolchain

DevOps Description

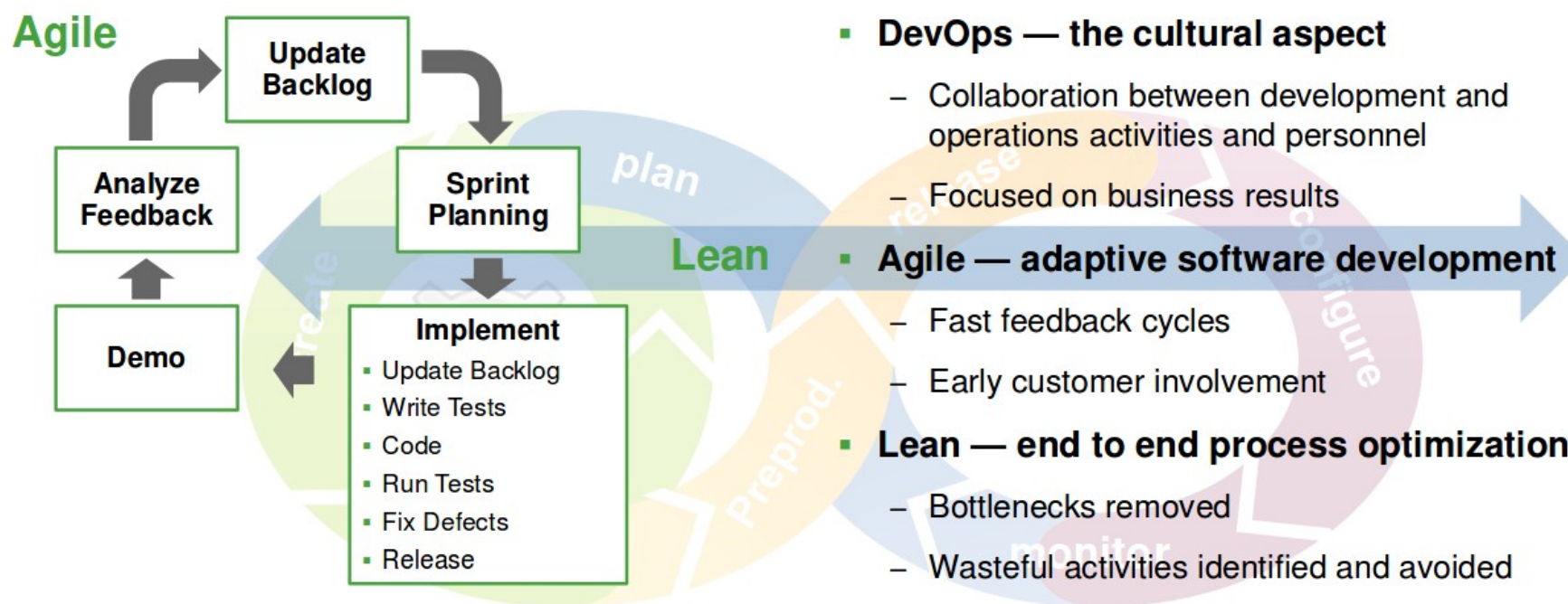


- DevOps is a perspective that requires cultural change and focuses on rapid IT service delivery through the adoption of agile, lean practices in the context of an integrated approach to deliver business value.
- DevOps emphasizes people and culture to improve collaboration between development and operations groups as well as other IT stakeholders such as architecture and information security.
- DevOps implementations utilize technology, especially automation tools that can leverage an increasingly programmable and dynamic infrastructure from a life cycle perspective.

Gartner DevOps Toolchain

DevOps Toolchain Supported by Agile Practices and Lean Principles

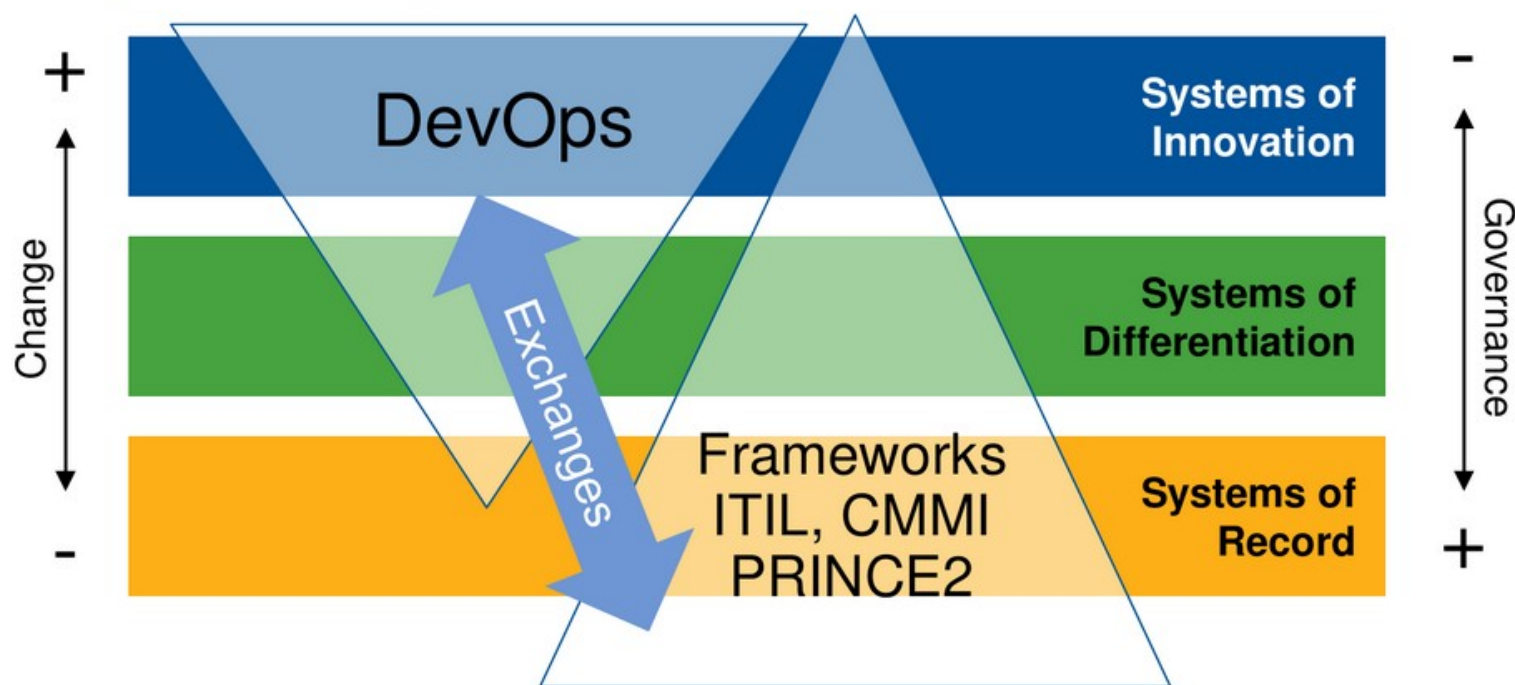
@ the core of change



Gartner Bimodal IT

Pace Layering and Bimodal IT Are Complementary

Step 3 @ the core of change



Pace classifies systems,
whereas bimodal IT is about capabilities

23 © 2016 Gartner, Inc. and/or its affiliates. All rights reserved.

Gartner

<http://gartner.daycohost.local/2016/Sessions%20Documents/SessionDocumentation/LSC35%20-%20A3%20-%20Seven%20Steps%20to%20Start%20Your%20DevOps%20Initiative%20-%2020310065.pdf>

Gartner DevOps vs. ITIL

There Is No Definitive Text

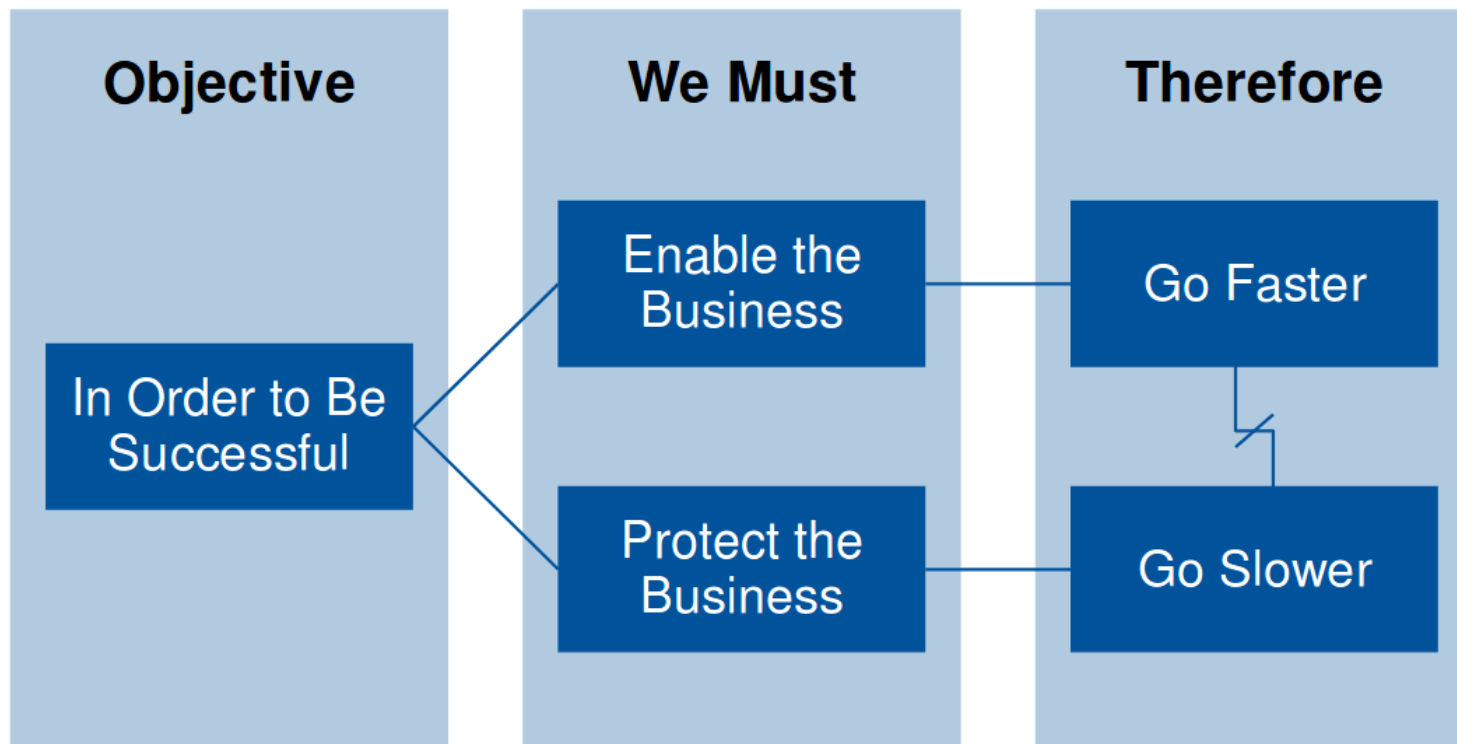
Step 4 @ the core of change

DevOps



El Origen de “Bimodal IT”

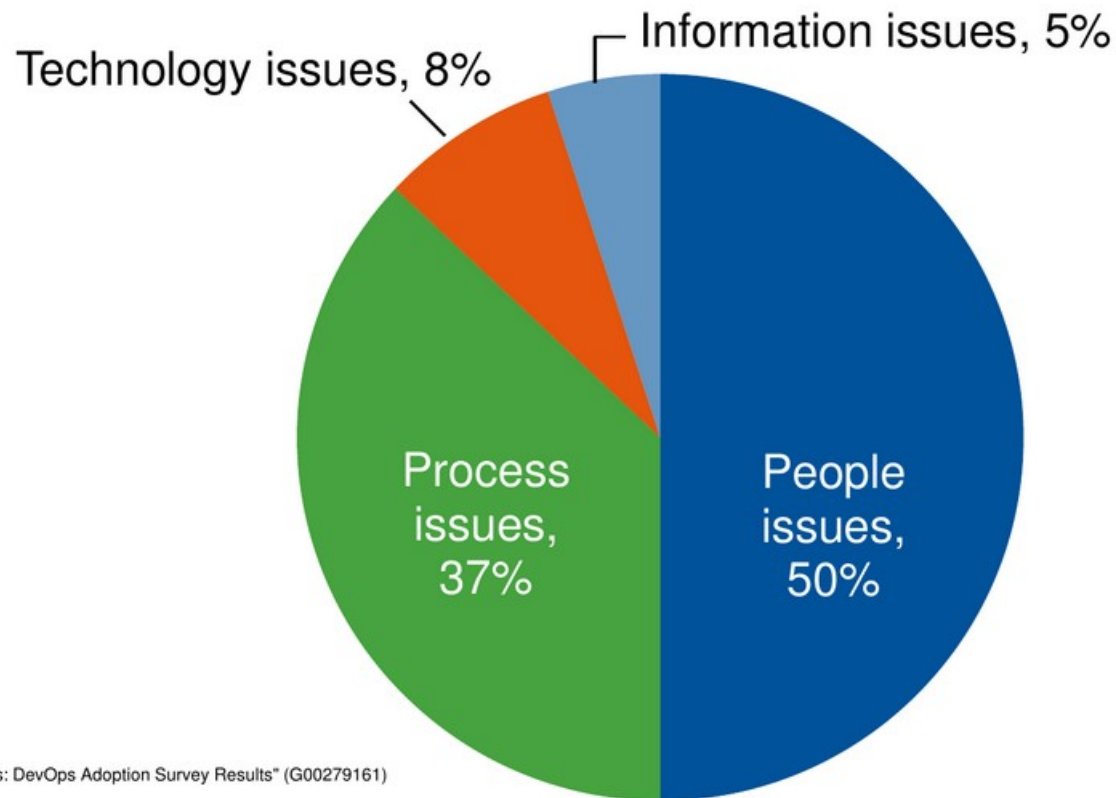
The Smoking Gun That Started the War



Gartner DevOps Adoption

People Are the Greatest Challenge

Step 2 @ the core of change



n = 113

Source: "Survey Analysis: DevOps Adoption Survey Results" (G00279161)

14 © 2016 Gartner, Inc. and/or its affiliates. All rights reserved.

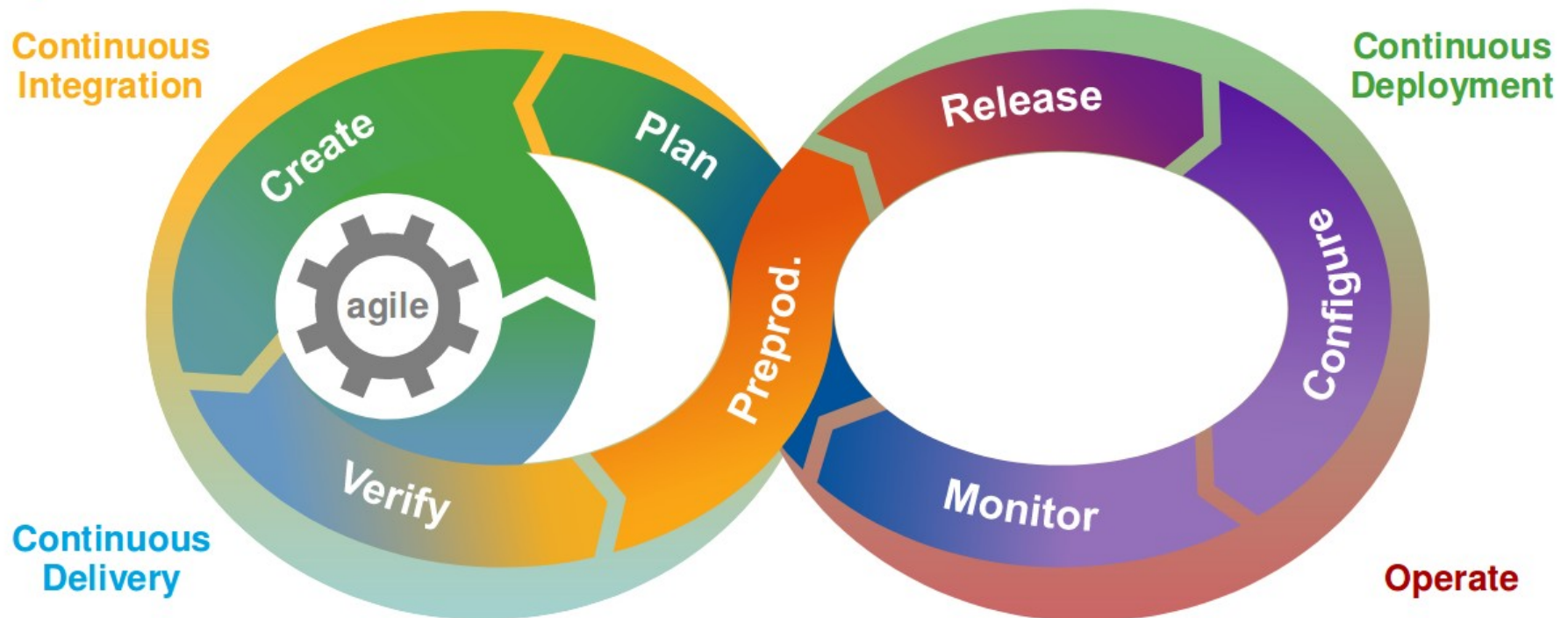
Gartner

<http://gartner.daycohost.local/2016/Sessions%20Documents/SessionDocumentation/LSC35%20-%20A3%20-%20Seven%20Steps%20to%20Start%20Your%20DevOps%20Initiative%20-%2020310065.pdf>

Gartner DevOps Toolchain

DevOps Toolchain Key Initiative Alignment

@ the core
of change



Gartner DevOps Toolchain

The Gartner DevOps Toolchain Methodology

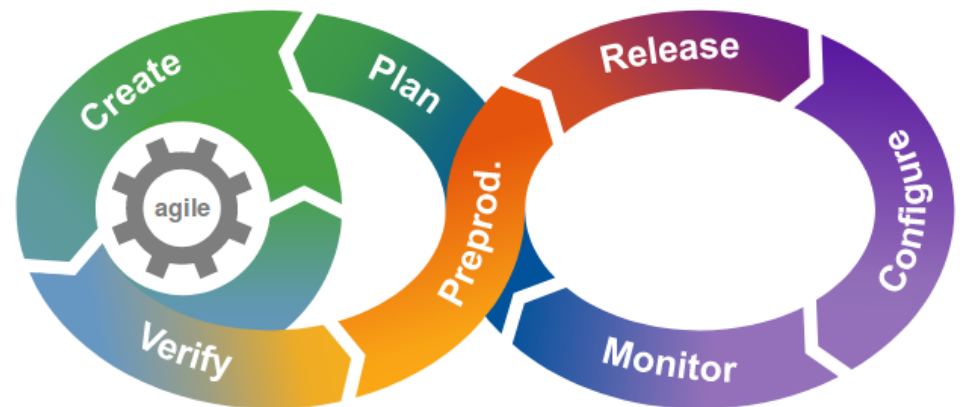


How to use it:

- Activity-based DevOps pipeline definition to help make tools choice and usage decisions
- Make alterations to the activities specific to your DevOps projects
- Identify tools integration and handoffs
- Understand the skills and resources required specific to the tools chosen

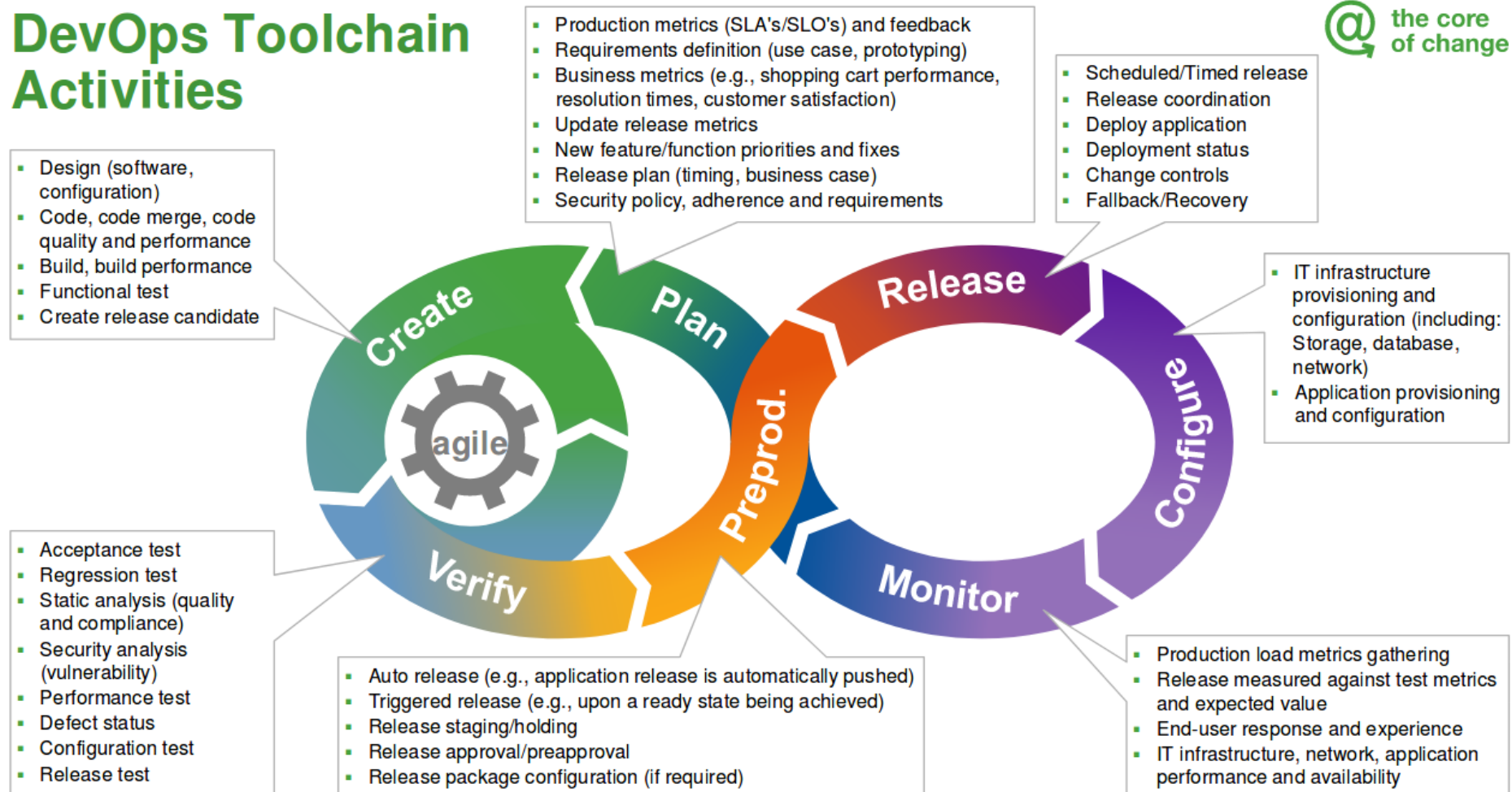
How **NOT** to use it:

- As a definitive list of DevOps activity that applies to all DevOps projects
- As a DevOps process map



Gartner DevOps Toolchain

DevOps Toolchain Activities



Gartner DevOps Toolchain

DevOps Tools



DevOps Ready

- Tools that remove constraints, enabling automated and parallel activities
- Tools designed and used by both development and operations

DevOps Enabled

- Tools that automate and orchestrate toolchain activities
- Tools that provide collaboration and knowledge sharing
- Tools that can provide DevOps value but have to be modified to do so

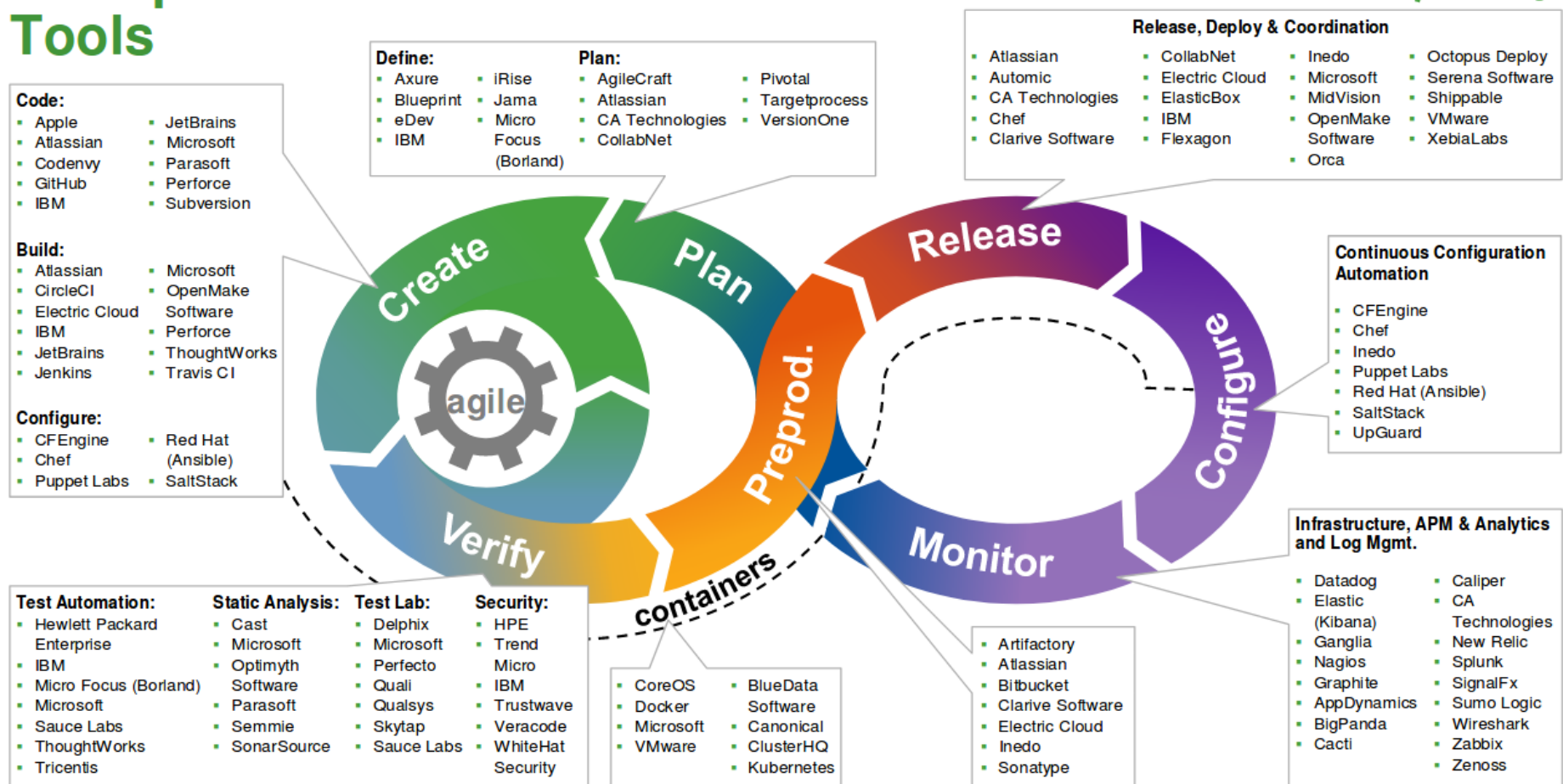
DevOps Capable

- Stand-alone tools that can work in a DevOps pipeline when configured correctly
- Tools that meet your specific DevOps needs

Gartner DevOps Toolchain

DevOps Toolchain Tools

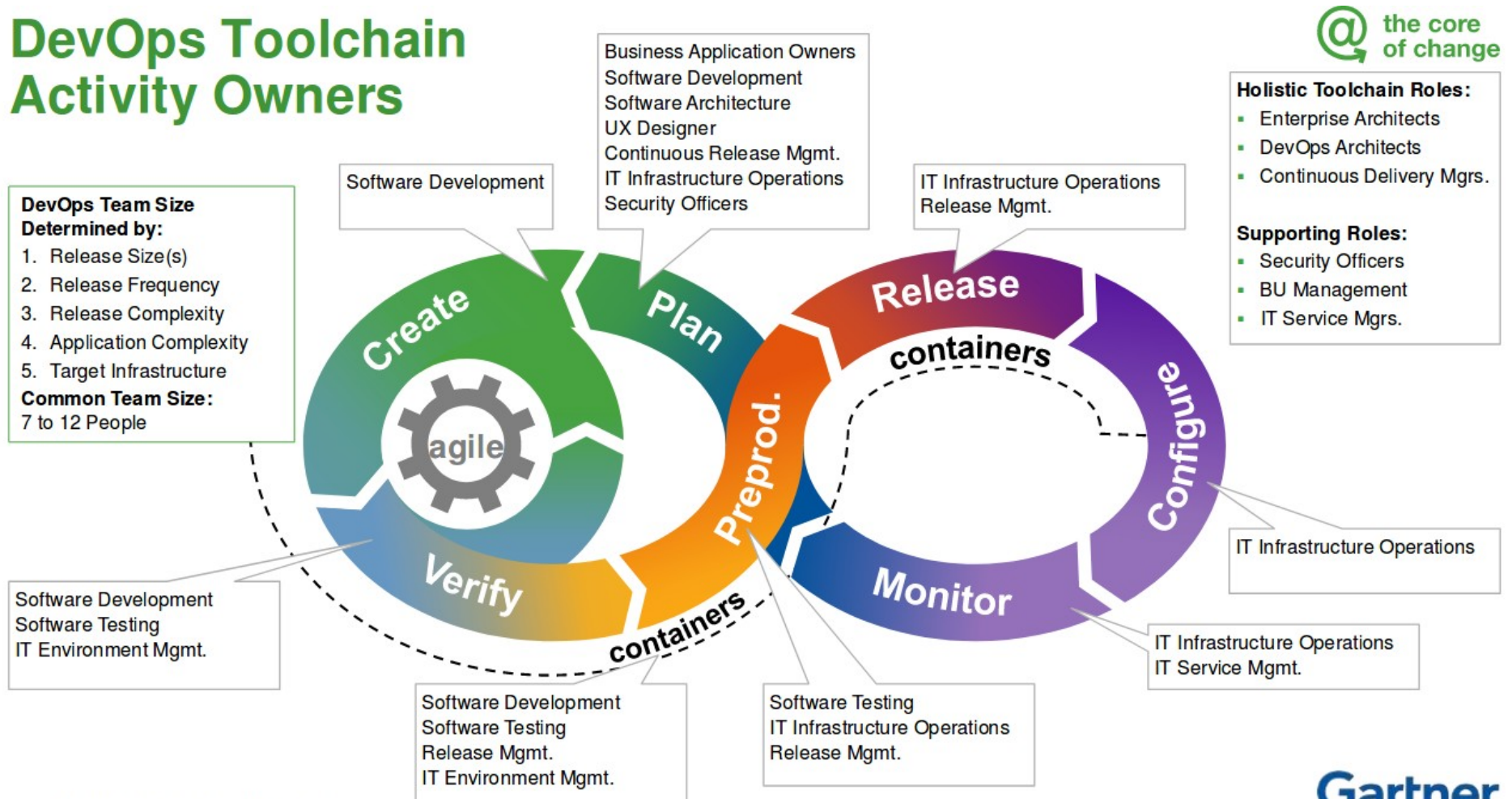
@ the core of change



<http://gartner.daycohost.local/2016/Sessions%20Documents/SessionDocumentation/LSC35%20-%20A5%20-%20Making%20Sense%20of%20DevOps%20Using%20the%20Gartner%20DevOps%20To%20-%2020310066.pdf>

Gartner DevOps Toolchain

DevOps Toolchain Activity Owners



21 © 2016 Gartner, Inc. and/or its affiliates. All rights reserved.

Gartner

<http://gartner.daycohost.local/2016/Sessions%20Documents/SessionDocumentation/LSC35%20-%20A5%20-%20Making%20Sense%20of%20DevOps%20Using%20the%20Gartner%20DevOps%20To%20-%2020310066.pdf>

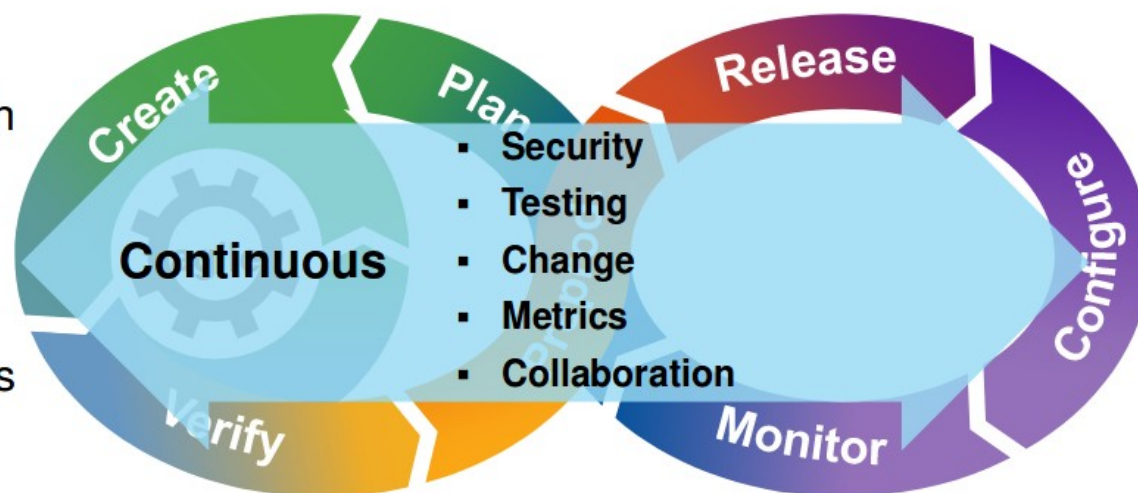
Gartner DevOps Toolchain

DevOps Toolchain Continuous Activities



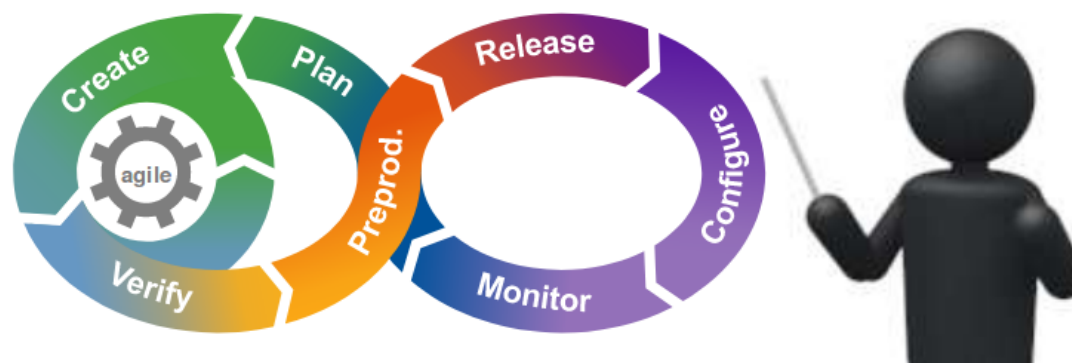
Continuous DevOps Toolchain Activities:

- Define how tools work together in support of the overall objective
- Describe where integration and handoffs are needed
- Show where automation provides the greatest value
- Promotes team pipeline awareness and fosters collaboration



Gartner DevOps Toolchain

DevOps Toolchain Orchestration Can One Tool Do It All?



Challenges:

- DevOps toolchains are often built from discrete and often disconnected tools.
- DevOps complexity is driving IT organizations to seek ways to get greater control and visibility over the entire DevOps process.



Findings:

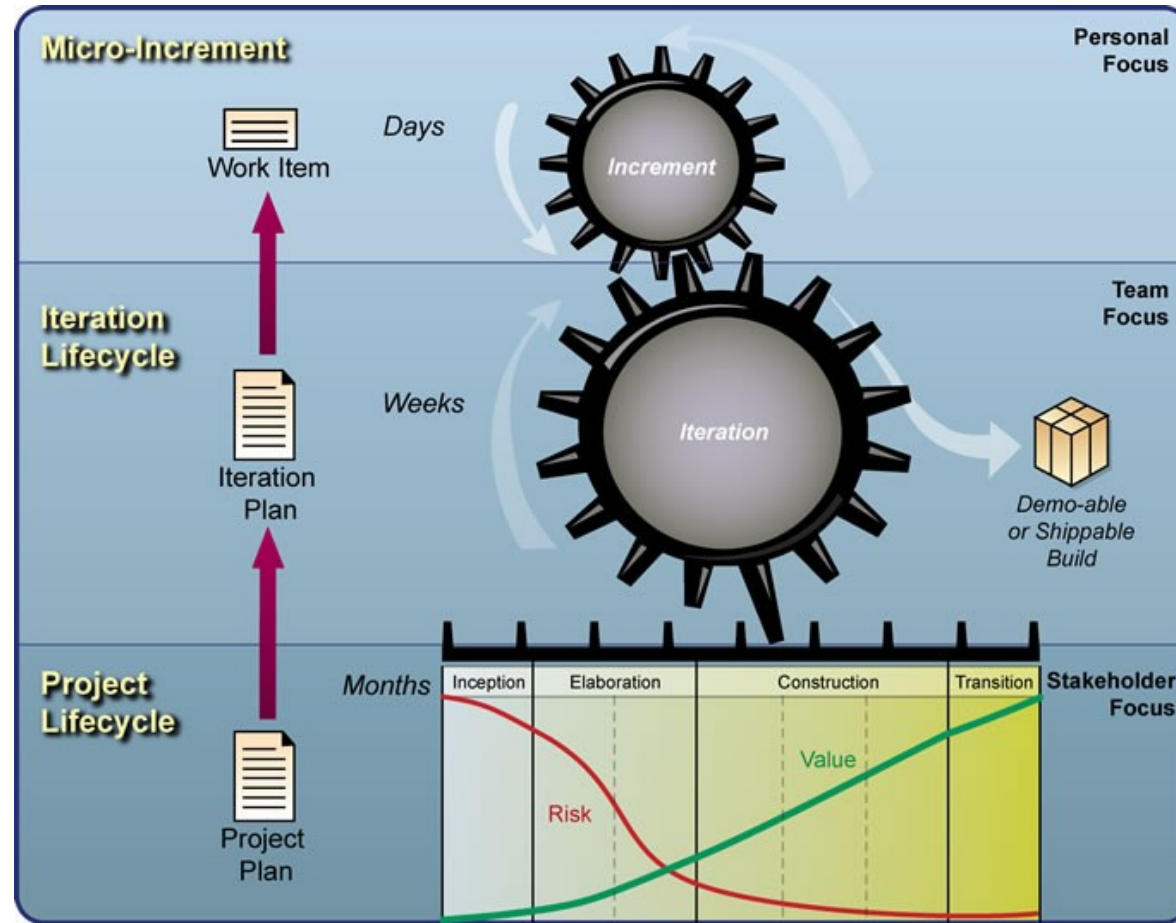
- Tools vendors tools claim they simplify the DevOps toolchain but none support the entire toolchain.
- Toolchain orchestration can be compelling but it can introduce different challenges.



Recommendations:

- Toolchain orchestration must be evaluated against entire DevOps objective and for each DevOps area.
- Quantify the trade-offs of centralizing or duplicating orchestration in your toolchain.

Open Unified Process (OpenUP)



Core Principles



Roles



Work Products



Disciplines



Lifecycle

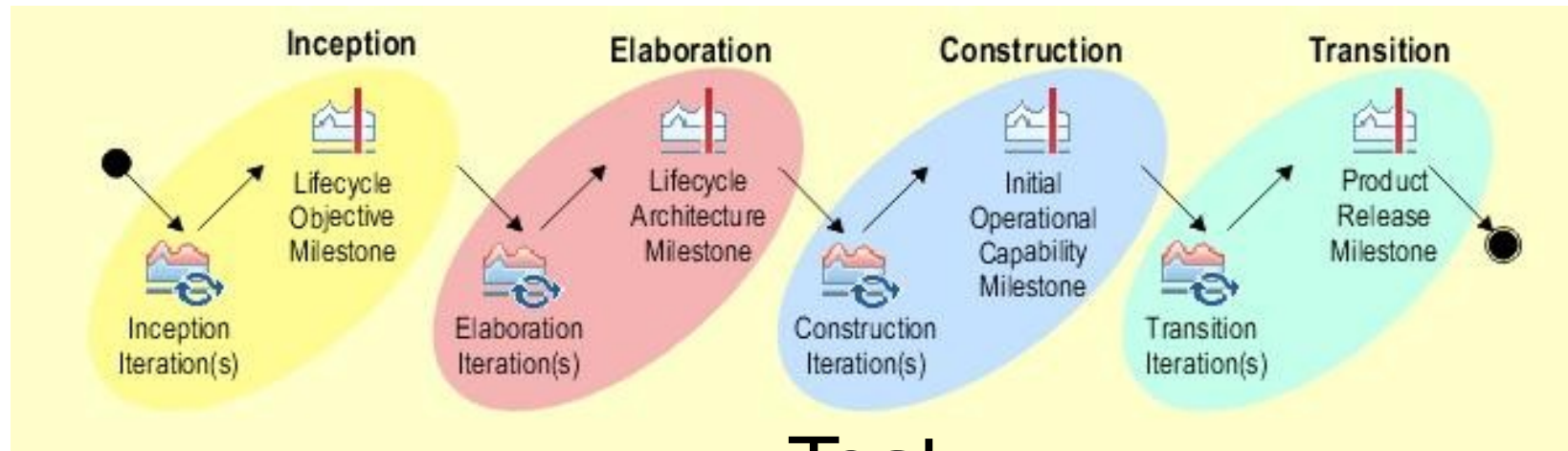


<http://epf.eclipse.org/wikis/openup/>

OpenUP: Principles

- ***Balancear las prioridades*** que compiten entre sí para maximizar el valor a los interesados.
- ***Colaborar para alinear intereses*** y compartir el entendimiento.
- ***Enfoque temprano en la arquitectura*** para minimizar riesgos y organizar el desarrollo
- ***Evolucionar para obtener retroalimentación*** constantemente y mejorar

OpenUP: Life-Cycle / Disciplines



- Requirements
- **Architecture**
- Environment
- Development
- Test
- Deployment
- **Project management**

OpenUP: Roles



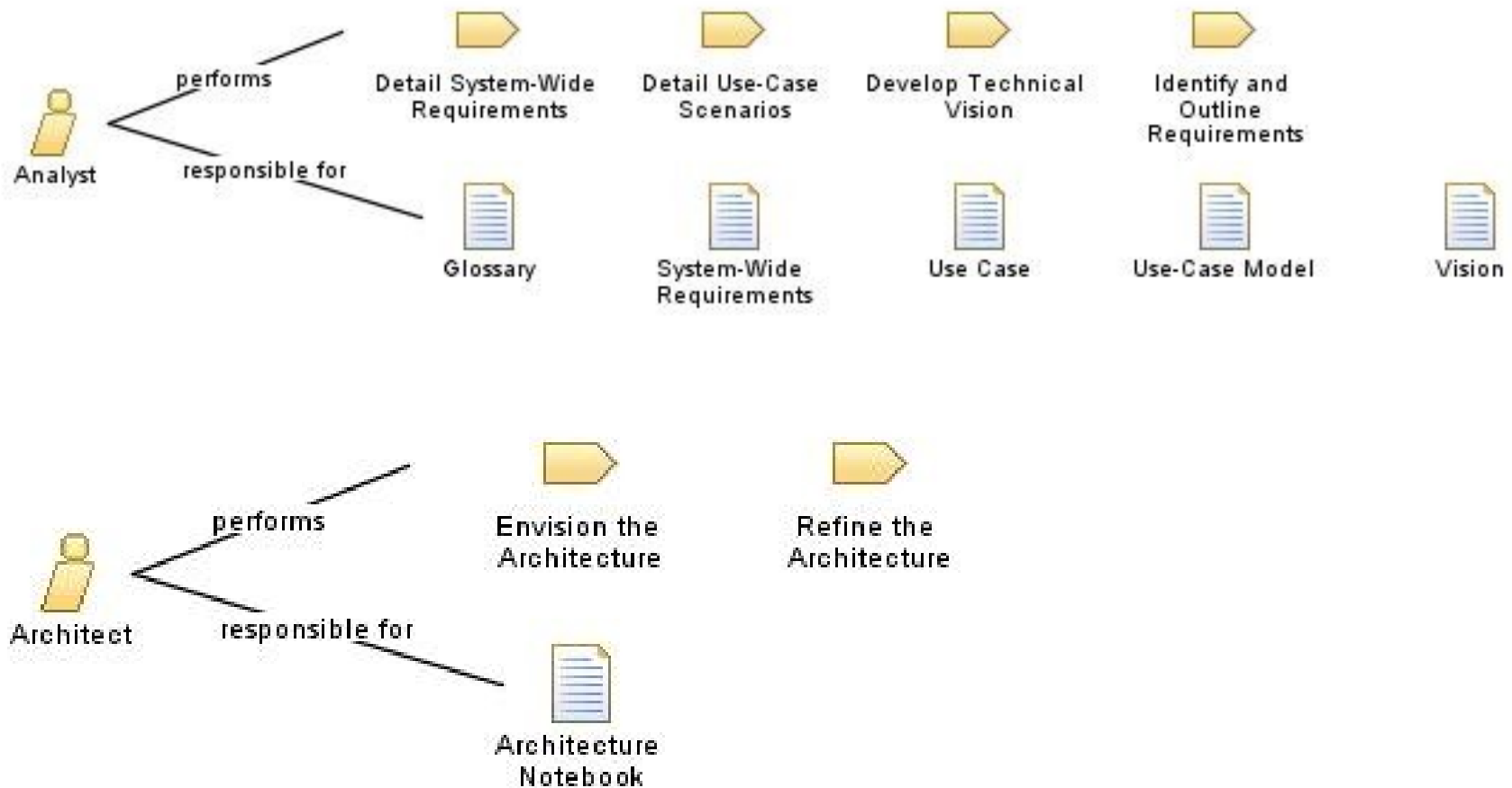
- **Basics**
 - Analyst
 - Architect
 - Developer
 - Project Manager
 - Stakeholder
 - Tester
- **Development**
 - Course Developer
 - Deployment Engineer
 - Deployment Manager
 - Product Owner
 - Technical Writer
 - Trainer
- **Enviroment**
 - Process Specialist
 - Tool Specialist

Incremento de Concurrency

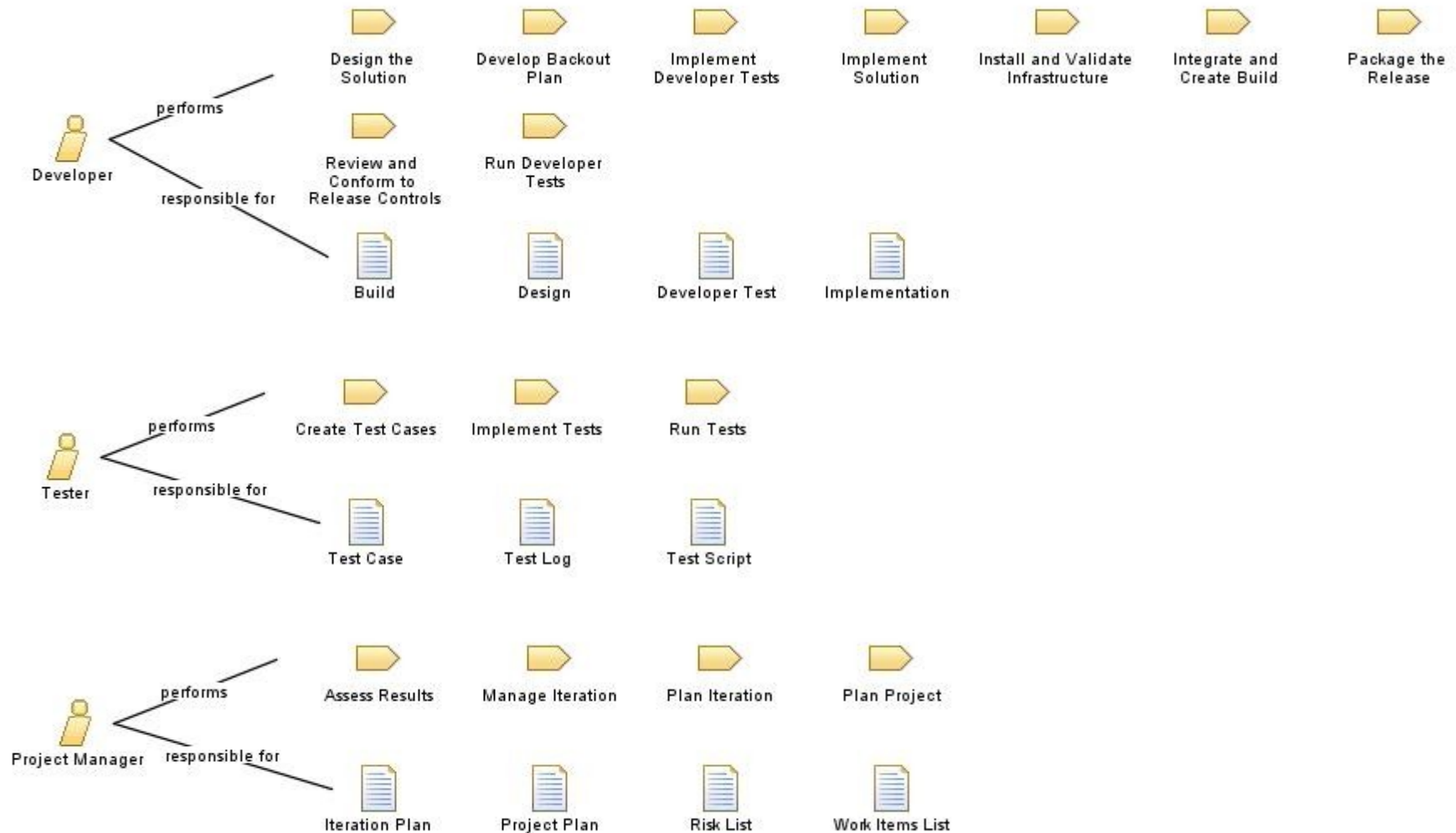
OpenUP: Work Products

Disciplines	Work Products
Requeriments	Glosary, Vision, System-Wide Requirements, Use Case Model, Use Cases
Architecture	Architecture Notebook
Enviroments	Project Defined Process, Tools
Development	Design , Implementation, Build, Developer Test
Test	Test Cases, Test Scripts, Test Logs
Deployment	Product Documentation , Support Documentation, User Documentation, Training Materials , Backout Plan, Deployment Plan, Infrastructure , Release Communications, Release Controls, Release
Project Management	Risk List, Work Items List, Iteration Plan, Project Plan

OpenUP: Work Responsibilities



OpenUP: Work Responsibilities



OpenUP: Practices

Management:

- ...
- **Project Process Tailoring**

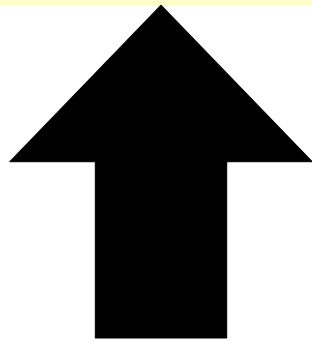
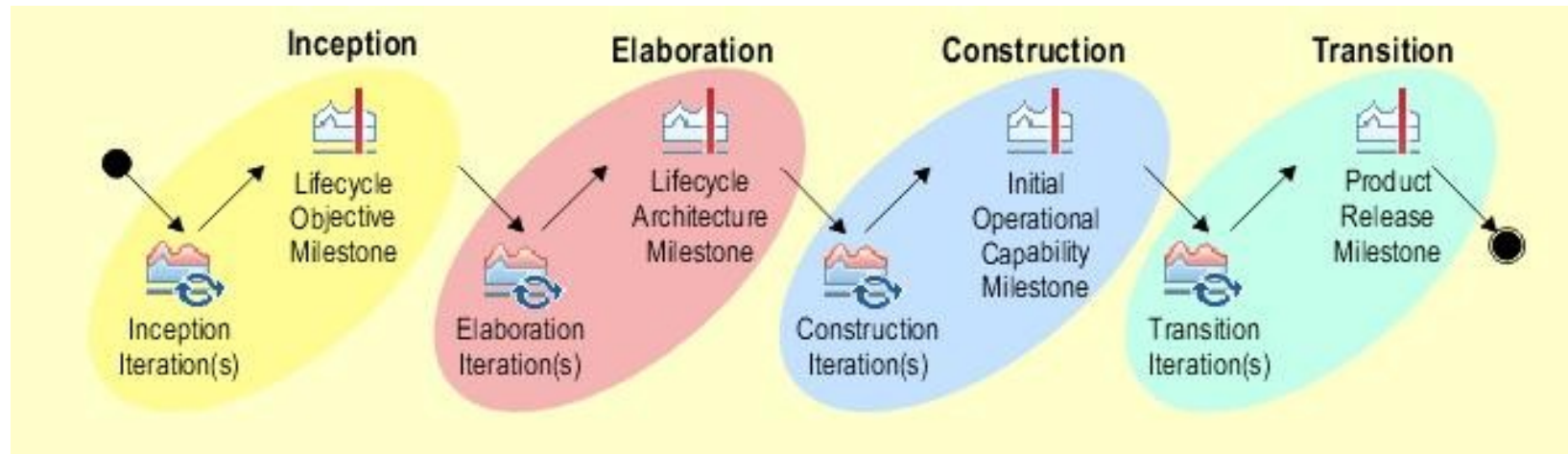
Deployment:

- Documentation and **training**
- ...

Technical:

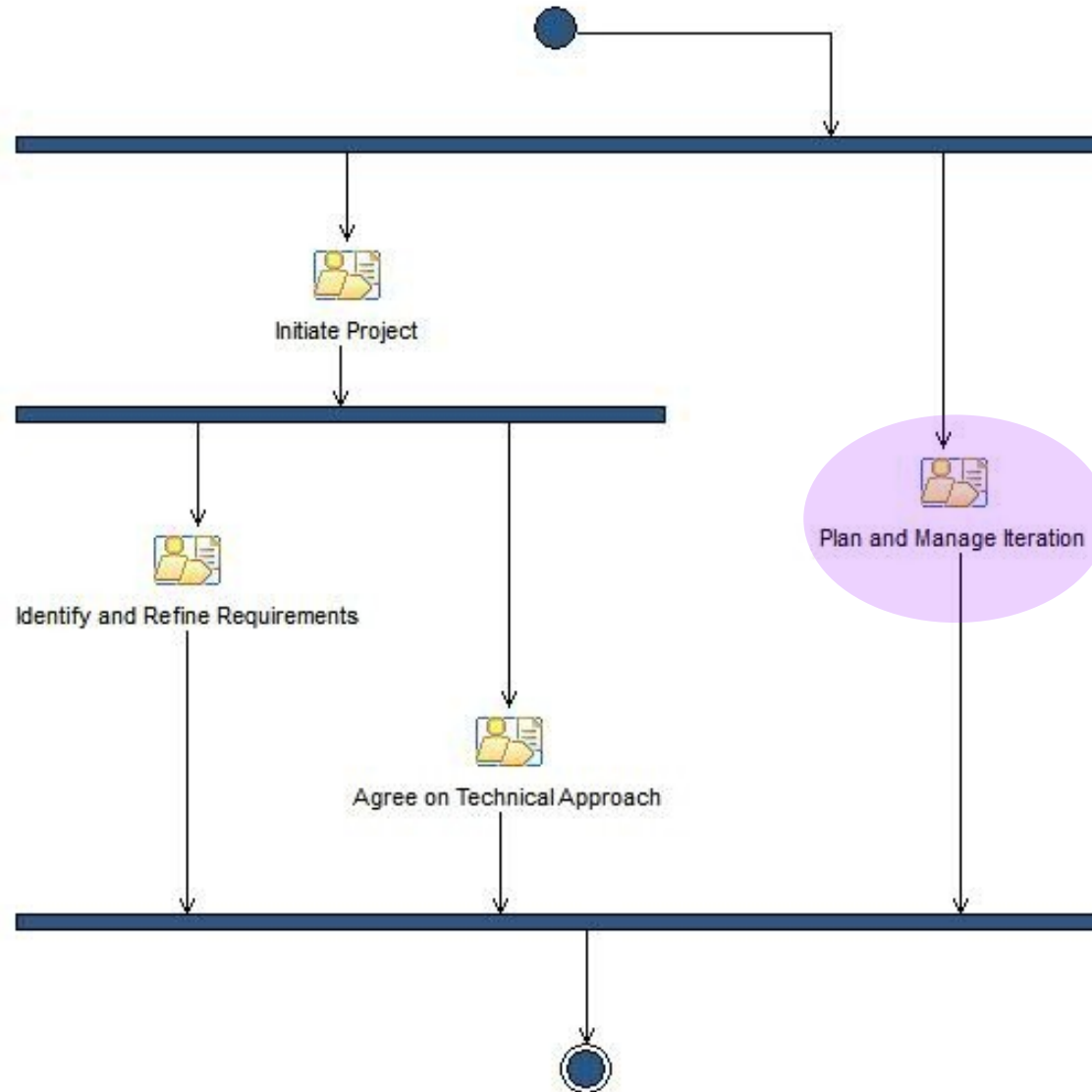
- ...
- **Continuous Integration**
- **Evolutionary Architecture**
- ...
- **Test Driven Development**
- **Use Case Driven Dev. Development**

OpenUP: Inception Phase

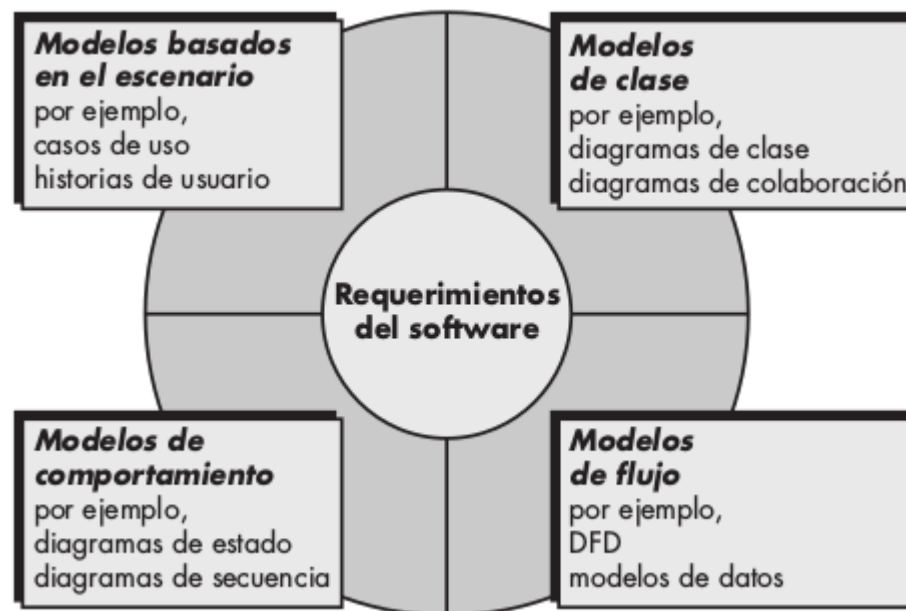
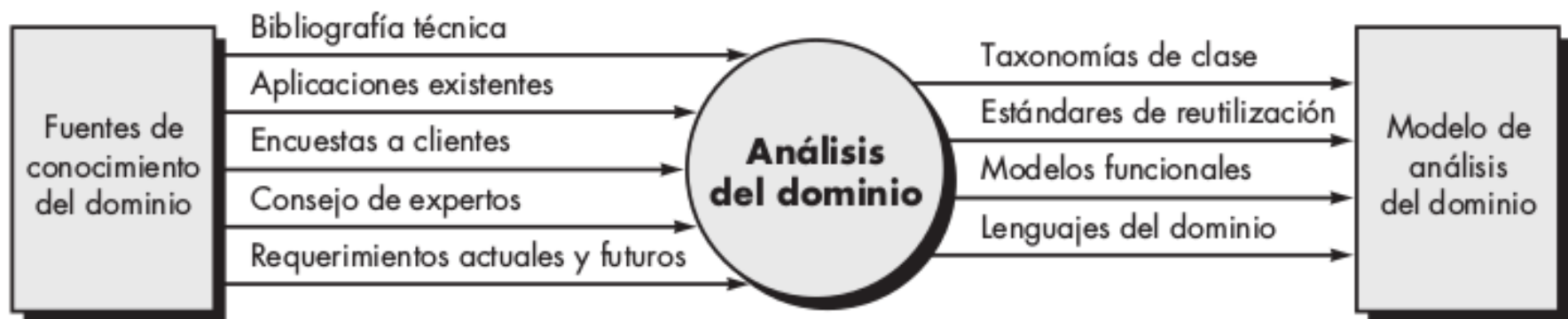


Inception

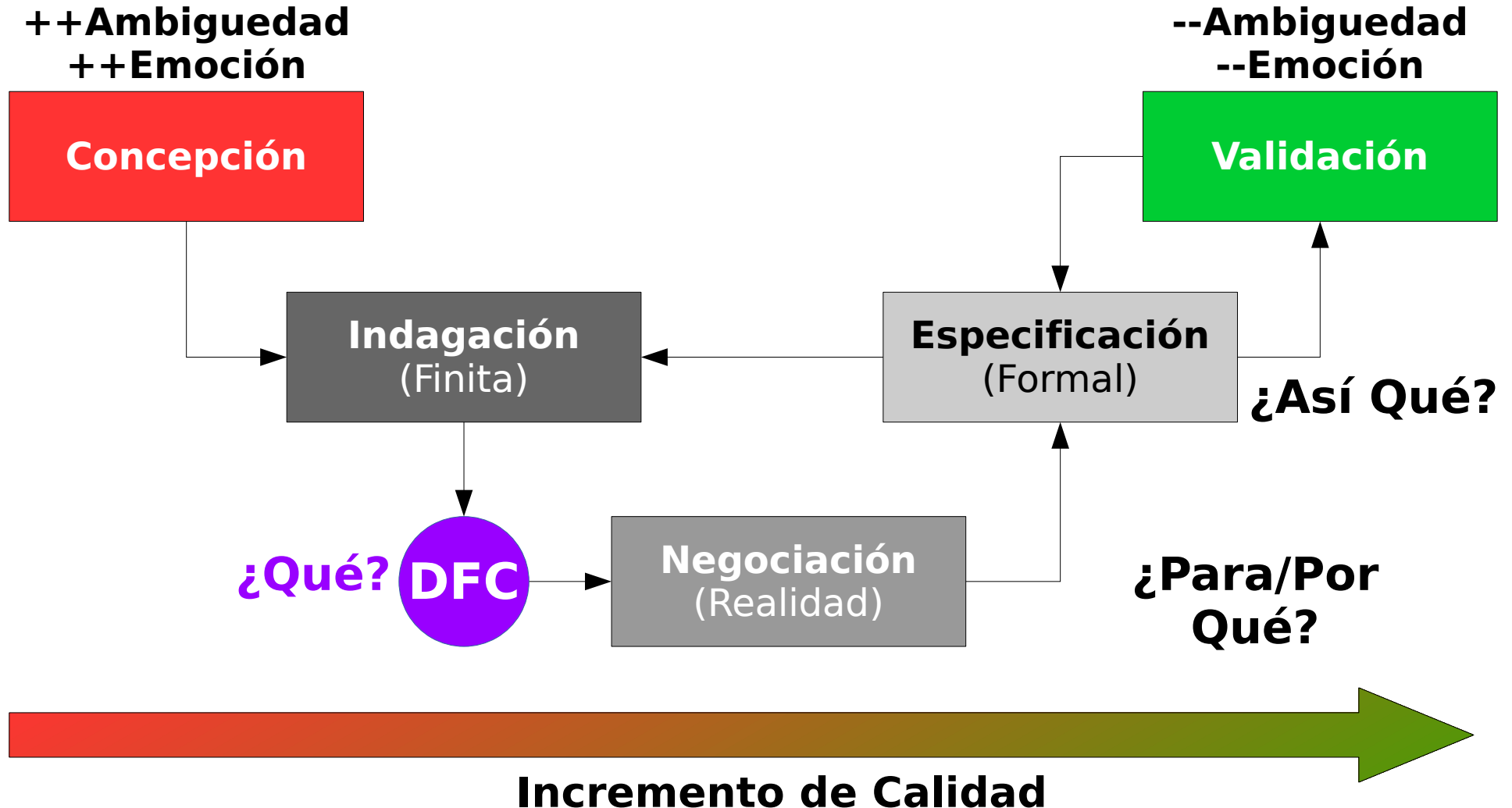
OpenUP: Inception Iter. WBS



Análisis de Requerimientos



Análisis de Requerimientos



Despliegue de la Función de Calidad (DFC)

- **Normales**

- Metas
- Objetivos

- **Esperados**

- Muy importantes
- No explícitos (Implícitos)
- Ausencia => Insatisfacción

- **Emocionantes**

- Deseables
- Explícitos
- Presencia => Mucha satisfacción

¿Qué?
~~¿Cómo?~~

Filtros Globales:

- Plan Estratégico de la Marca
- Vision Estratégica de Monitoreo
- Plan de Ejecución de Monitoreo

Especificación de Requerimientos

Entregables (narrativas y diagramas) dependiendo del dominio y complejidad de los requerimientos:

- **Escenarios:**
 - Casos de Uso
 - Actividades
- **Clases** (de Objetos)
- **Colaboración** (entre Clases)
- **Comportamientos** (de Clases)
 - Estados
 - Secuencias
- **Flujos de:**
 - Datos
 - Control

¿Qué?
~~¿Cómo~~
?

Modelado del Comportamiento (UML)

- **Diagrama de casos de uso:** ofrecen una visión general de los actores involucrados en un sistema, las diferentes funciones que necesitan esos actores y cómo interactúan estas diferentes funciones.
- **Diagrama de actividades:** describen el flujo de trabajo de cualquier componente de un sistema.
- **Diagrama de estados (máquina de estados):** se usan para describir el comportamiento de los objetos que actúan de manera diferente de acuerdo con su estado en un momento dado.

Modelado de la Estructura (UML)

- **Diagrama de clases:** muestra las clases en un sistema (orientado a objetos), atributos y operaciones y la relación entre cada clase.
- **Diagrama de componentes:** muestra la relación estructural de los componentes de un sistema de software complejo que tienen muchos componentes que se comunican entre sí mediante interfaces.
- **Diagrama de paquetes:** muestra las dependencias entre diferentes paquetes (conjunto de componentes) de un sistema.
- **Diagrama de despliegue:** muestra el hardware de su sistema y el software de ese hardware. Son útiles cuando la solución de software se despliega en varios equipos, cada uno con una configuración única.
- **Diagrama de objetos (diagramas de instancia):** similares a los diagramas de clases. muestran la relación entre los objetos, pero usan ejemplos del mundo real y en un momento dado.

Modelado de la Interacción (UML)

- **Diagrama global de interacciones:** similar a los diagramas de actividad muestran una secuencia pero de diagramas de interacción (7 tipos).
- **Diagrama de secuencia:** muestran cómo los objetos interactúan entre sí y el orden en que se producen esas interacciones.
- **Diagrama de colaboración (comunicación):** similar a los diagramas de secuencia, pero el foco está en los mensajes pasados entre objetos.
- **Diagrama de tiempos (sincronización):** similar a los diagramas de secuencia. Representan el comportamiento de los objetos en un marco de tiempo dado.

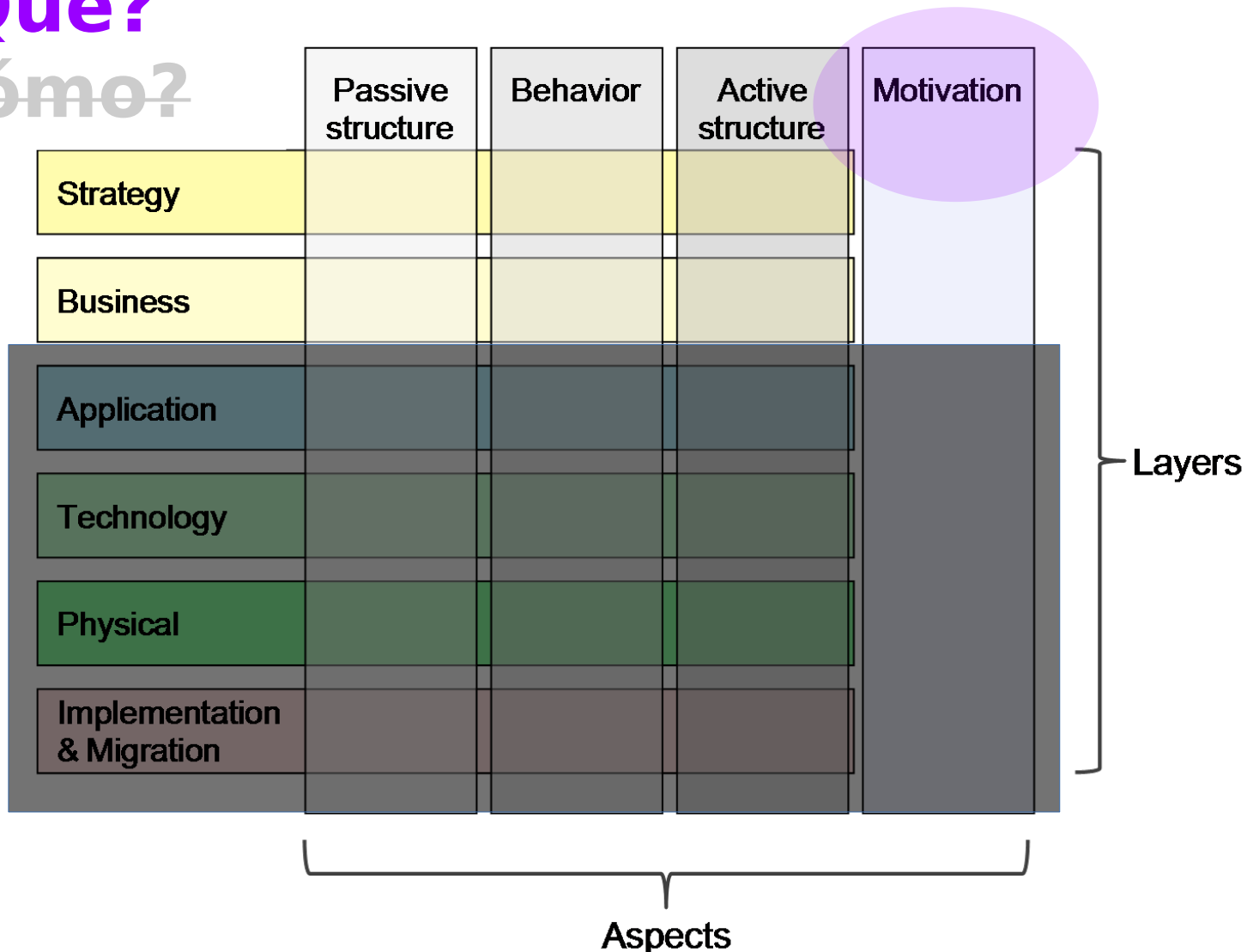
Validación de Requerimientos

- **Calidad:** Correctos, **Claros** (Exactos, Precisos, Sin ambigüedad), Completos, Consistentes, **Verificables**, Trazables, Posibles, **Independientes del diseño** y Atómicos.
- **Especificación:** funcionales (verbos), no funcionales (sustantivos), propios (componentes) y externos (interfaces), restricciones (“constraints”), estándares y regulaciones.
- **Clasificación mínima** (FURPS+OpenUP): **Functionality**, **Usability**, Scalability, Availability, **Reliability**, **Performance**, **Security**, Flexibility, Interoperability, Manageability, Maintainability, Supportability.

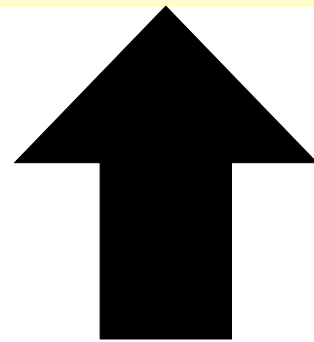
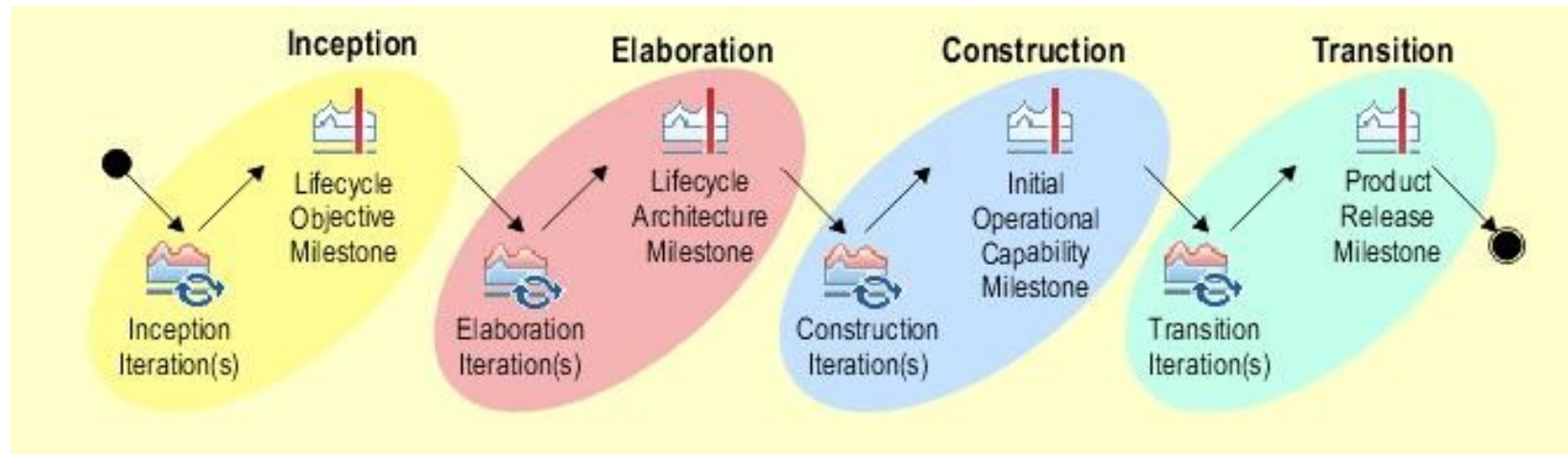
Motivación de la Arquitectura (TOGAF)

¿Qué?

¿Cómo?

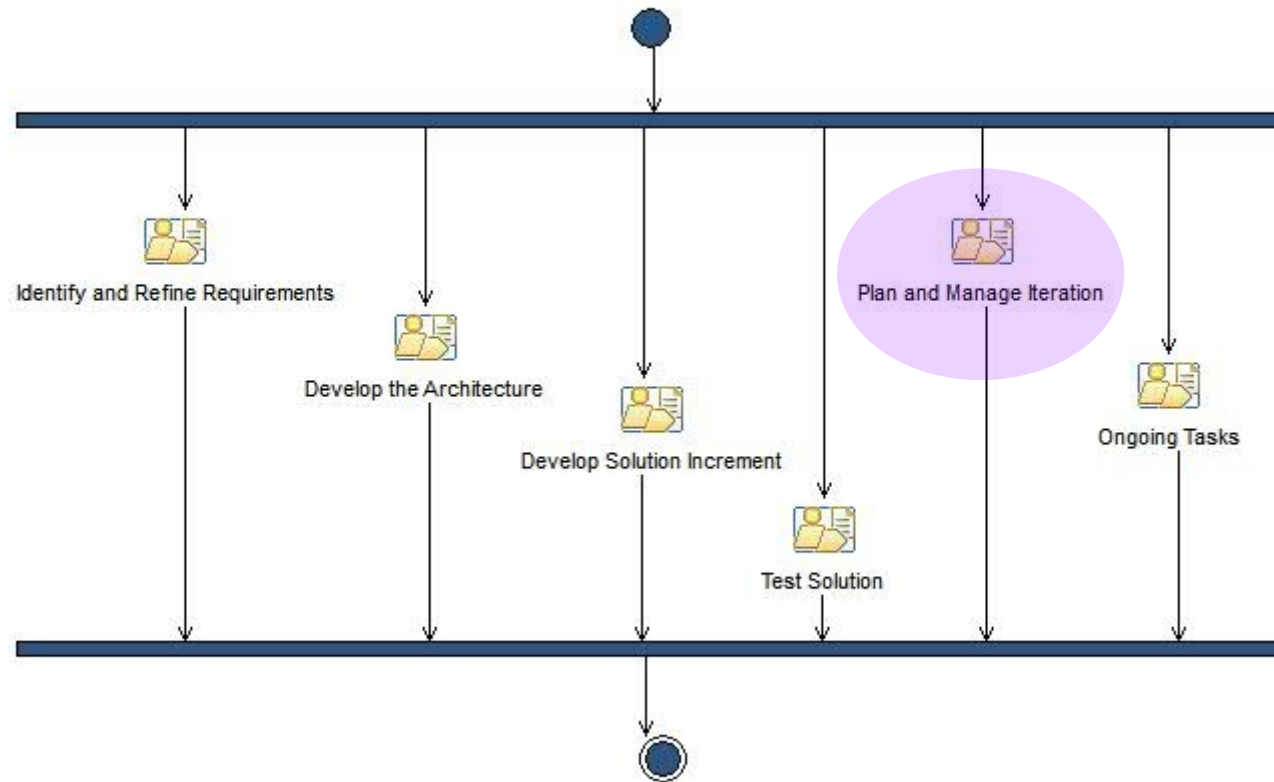


OpenUP: Elaboration Phase

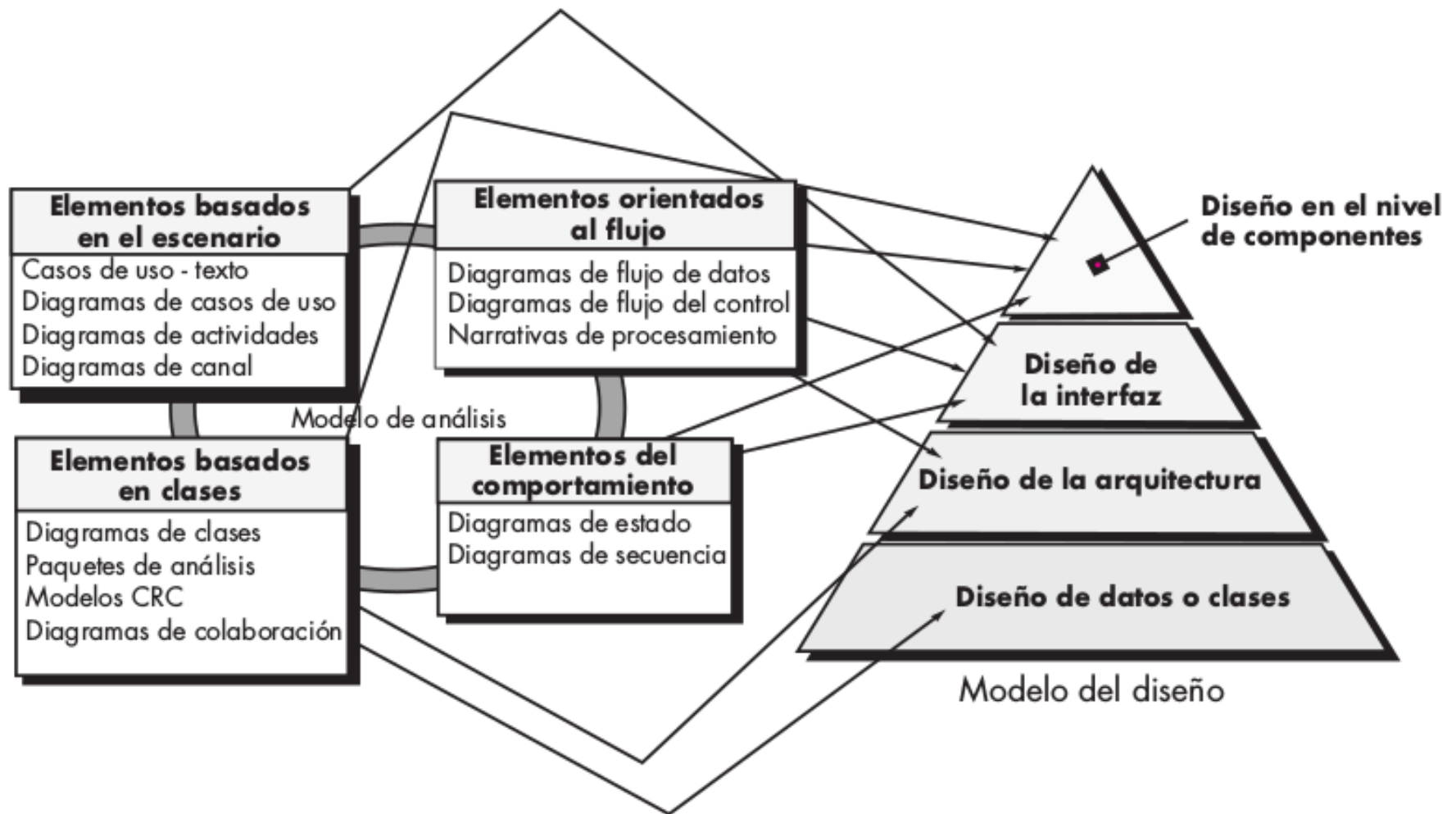


Elaborati on

OpenUP: Elaboration WBS



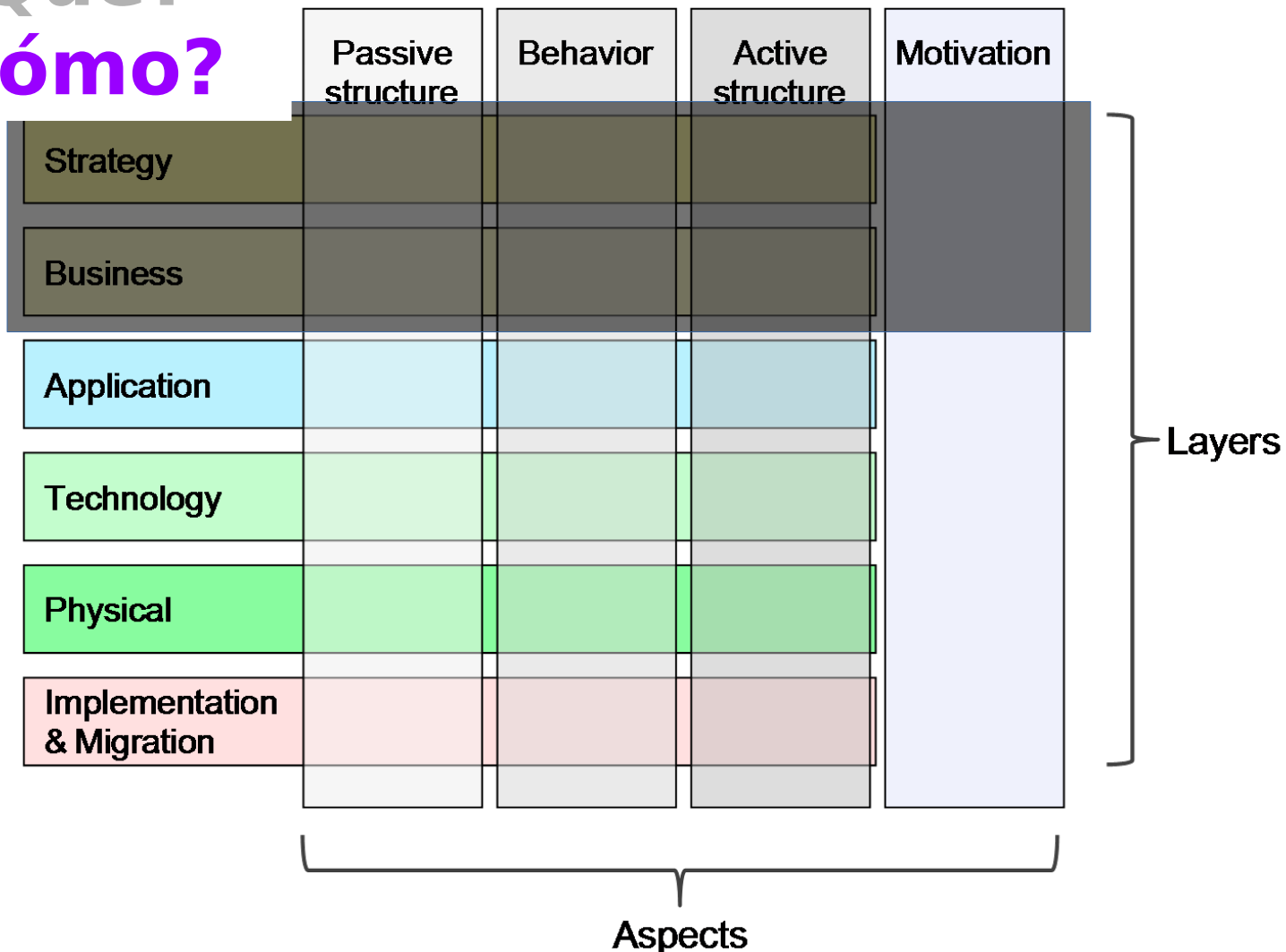
Elaboration: Analysis → Design



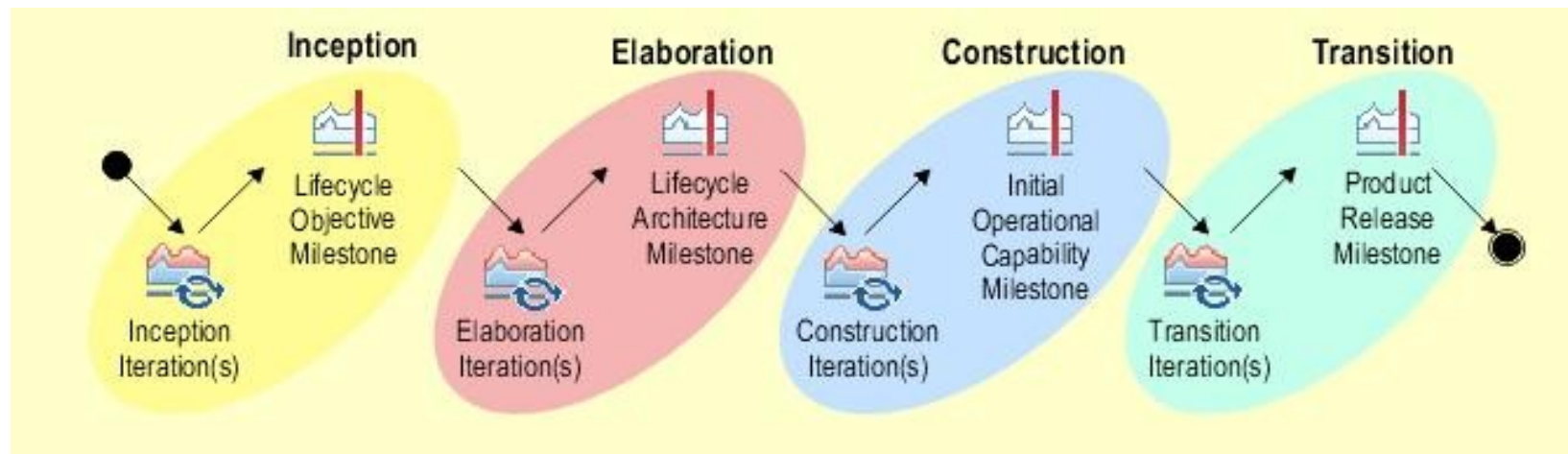
CRC = Clase / Responsabilidad / Colaboración

Elaboration: Architecture (TOGAF)

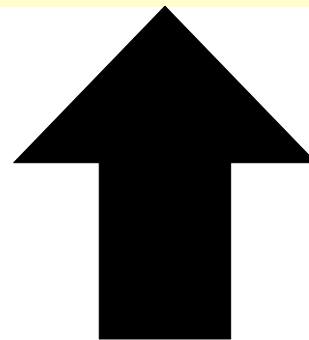
~~¿Qué?~~
¿Cómo?



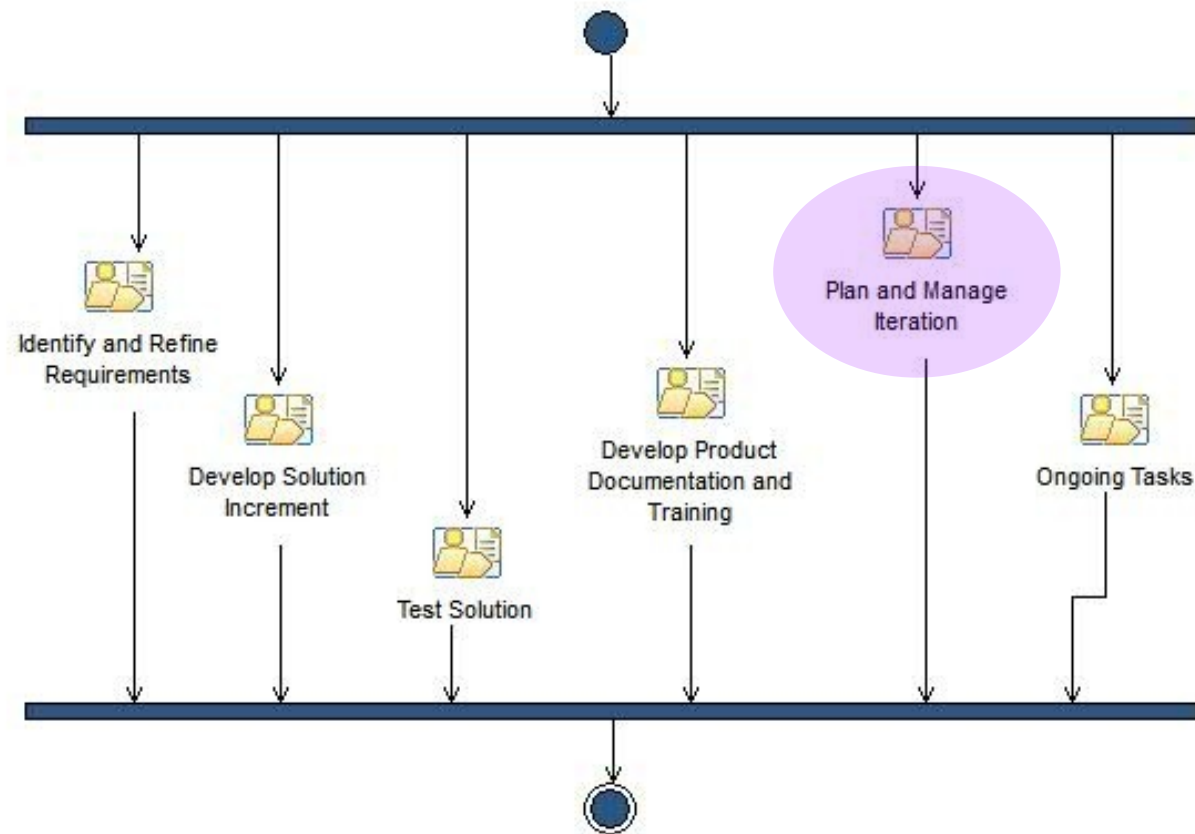
OpenUP: Construction Phase



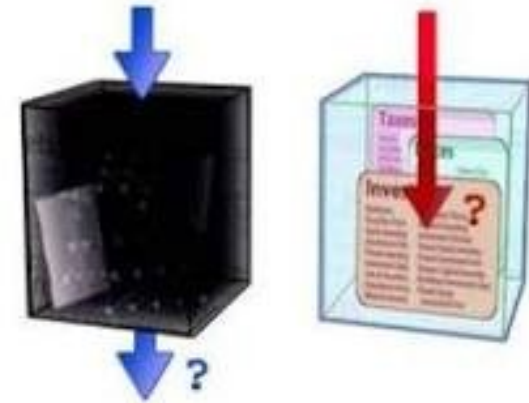
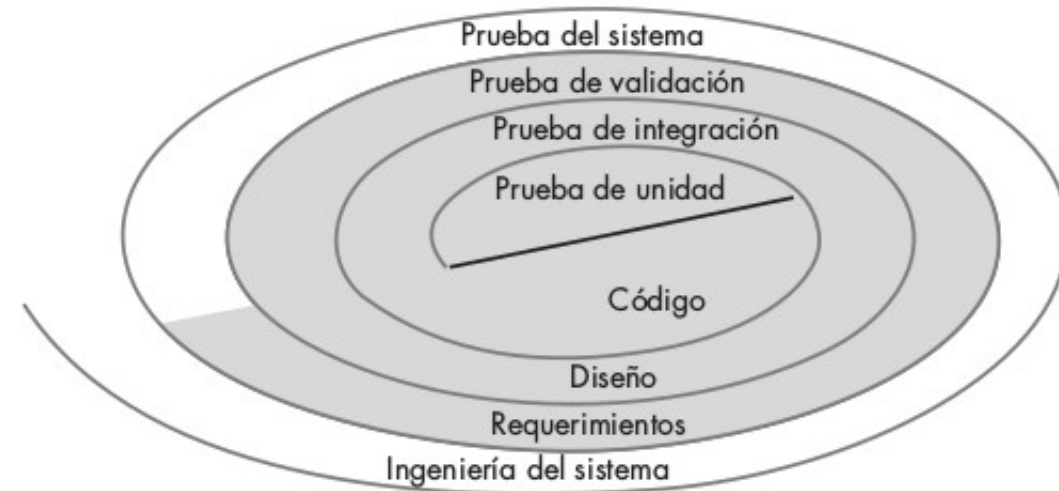
**Constructi
on**



OpenUP: Construction WBS



OpenUP: Estrategia de Pruebas

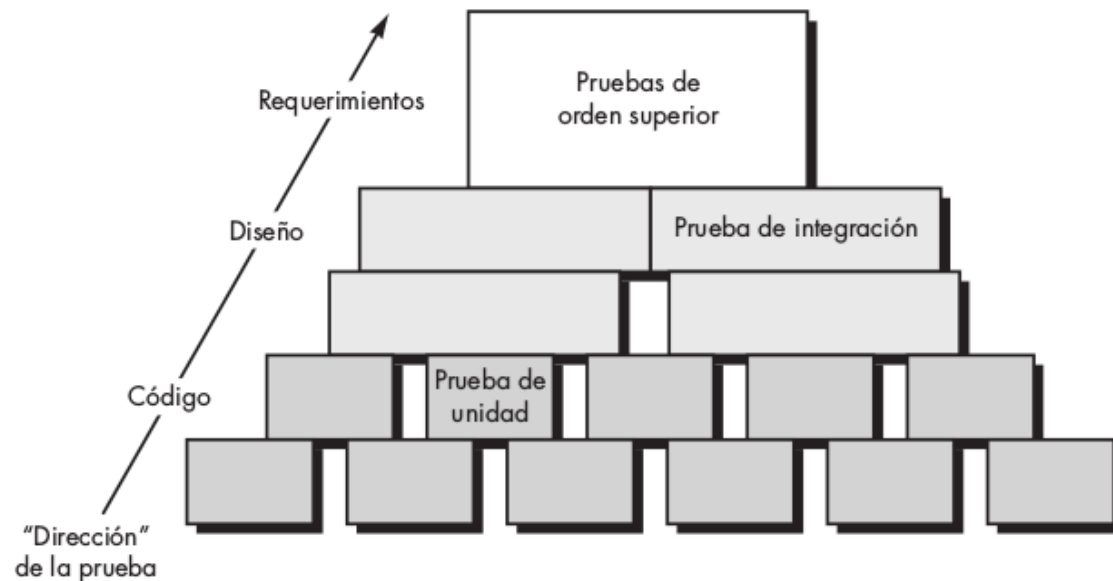


Pruebas de verificación:

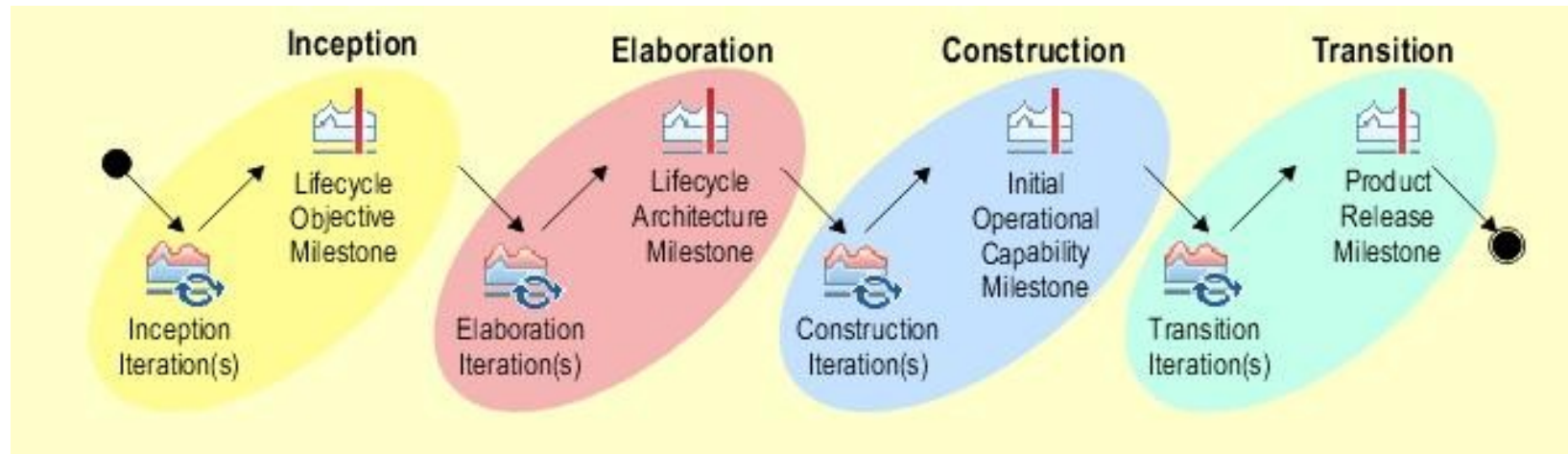
- Unitarias (Unit Frameworks)
- Caja Negra (Input/Output)
- Caja Blanca (Glass)
- Integración

Pruebas de validación:

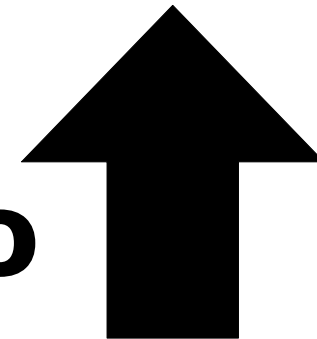
- **Alfa / beta** sobre un grupo de usuarios finales:
 - Alfa con el Desarrollador "Sobre el hombro".
 - Beta sin el Desarrollador



OpenUP: Transition Phase



**Transitio
n**



OpenUP: Transition WBS

